

경북대학교 AI·BigData 전문가 양성과정

## Module 13.

# 웹 기반 AI 서비스

경북대학교 배준현 교수

(joonion@knu.ac.kr)



### 학습 목표

이번 과정에서는 다음 목표를 달성한다.

- Flask/FastAPI 프레임워크를 이해하고 활용할 수 있다.
- Streamlit을 이용하여 웹 UI를 개발할 수 있다.
- Frontend와 Backend를 분리하여 개발할 수 있다.
- Hugging Faces를 활용한 AI 기반의 웹 서비스를 개발할 수 있다.
- LangChain을 이용하여 LLM 기반의 파이프라인을 구축할 수 있다.
- PDF 문서 기반 RAG Web Service를 개발할 수 있다.



### 필요한 패키지 설치

#### 패키지

#### 용도

#### 설치

*Flask*

파이썬 웹 프레임워크

```
python -m pip install flask
```

*FastAPI*

REST API 서버

```
python -m pip install fastapi
```

*Uvicorn*

FastAPI 실행 서버

```
python -m pip install uvicorn
```

*HTTPIe*

HTTP 요청 테스트

```
python -m pip install httpie
```

*Streamlit*

웹 UI 프레임워크

```
python -m pip install streamlit
```

# 경북대학교 AI·BigData 전문가 양성과정

## Lecture 1.

### World Wide Web and Flask Basics





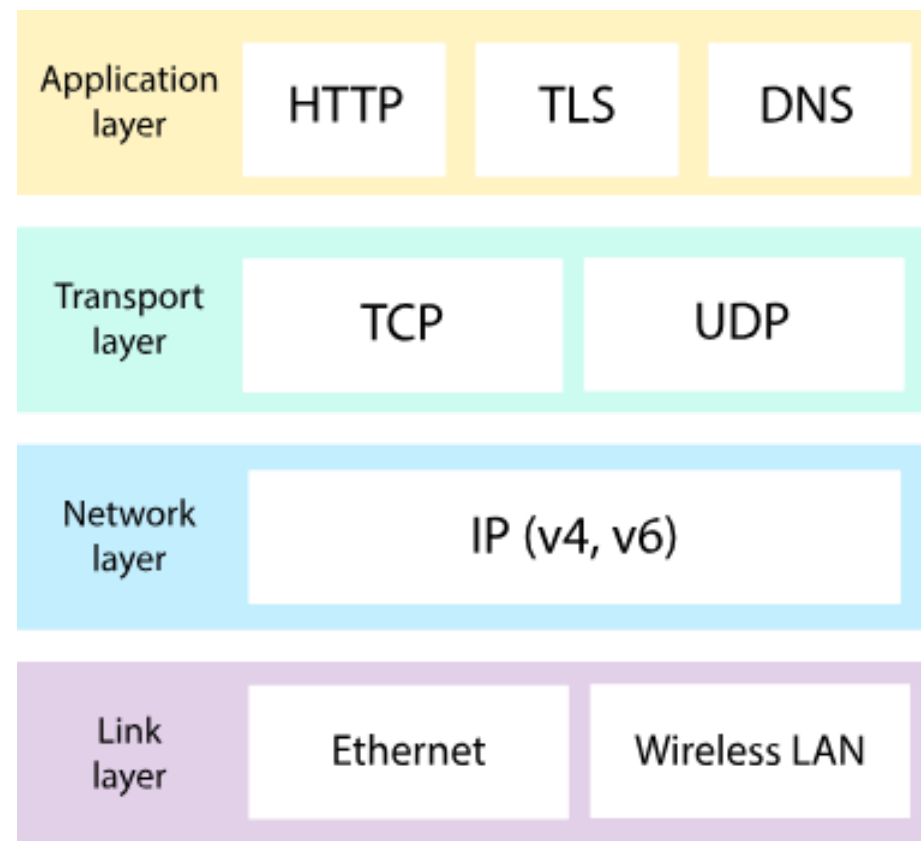
### 인터넷: Internet

인터넷이란, 전 세계의 컴퓨터들이 서로 연결된 네트워크를 말한다. (*inter-net*working)

인터넷에서 컴퓨터들은

- IP 주소
- TCP/IP 프로토콜

을 이용하여 서로 통신한다.





### IP 주소와 도메인 네임

IP (*Internet Protocol*) : 인터넷에 연결된 컴퓨터의 주소

*Domain Name*: 사람이 이해하기 쉽도록 IP 주소에 부여한 이름

IP Address

155.230.35.169

127.0.0.1

Domain Name

www.joonion.ac.kr

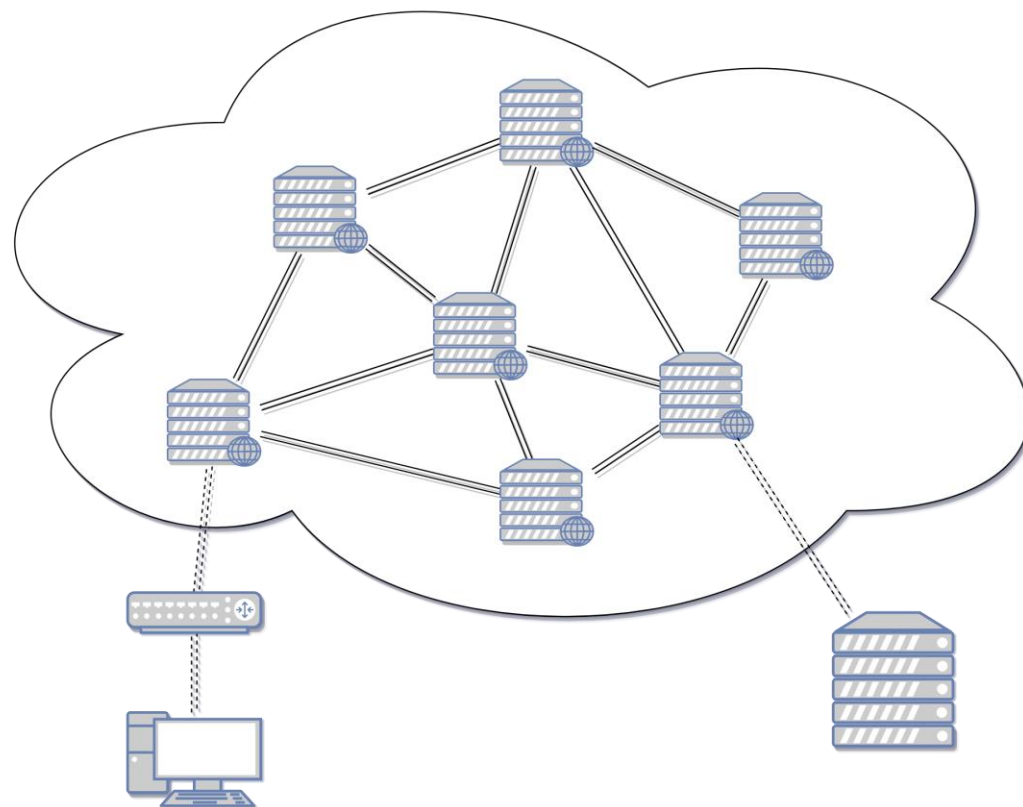
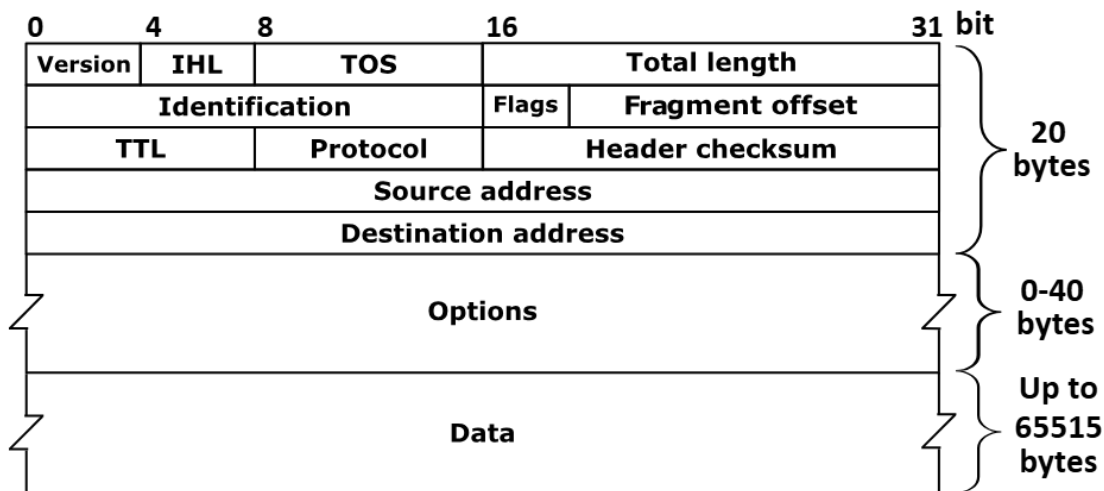
localhost



## 패킷과 라우팅

*Packet*: 인터넷에서 데이터를 전송할 때 사용하는 작은 데이터 단위

*Routing*: 패킷이 목적지까지 이동하는 경로를 결정하는 과정





### WWW과 HTTP

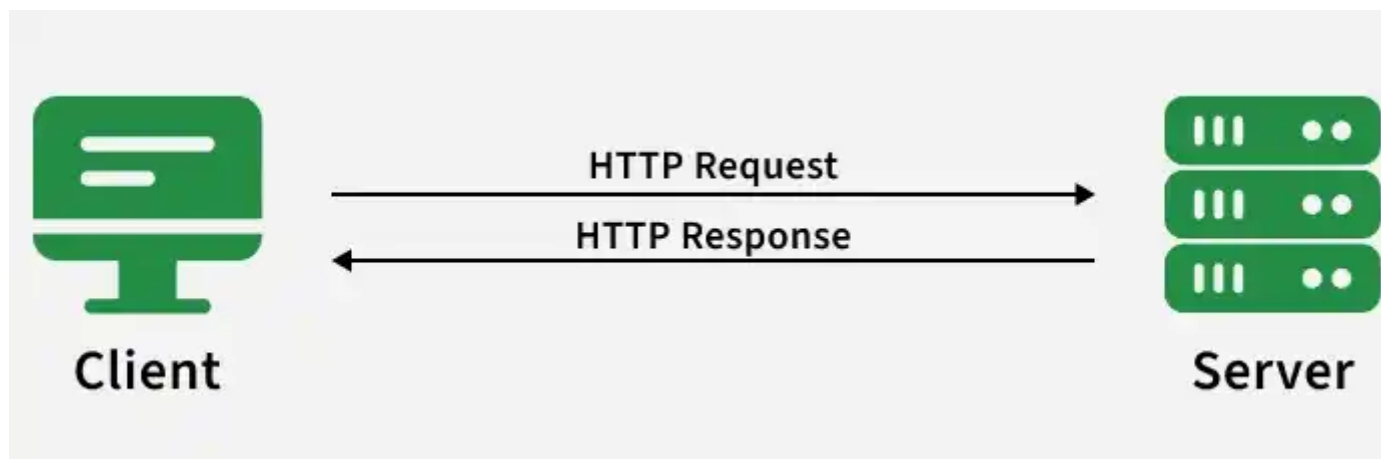
WWW: World Wide Web

*Hyperlink*를 통해 웹 페이지(문서)와 웹 페이지(문서)를 연결하는 네트워크

*HTTP*: HyperText Transfer Protocol

웹 브라우저와 웹 서버가 서로 문서를 주고 받는 통신 규약(*protocol*)

클라이언트가 요청(*request*)을 보내면 서버가 응답(*response*)을 반환해주는 구조

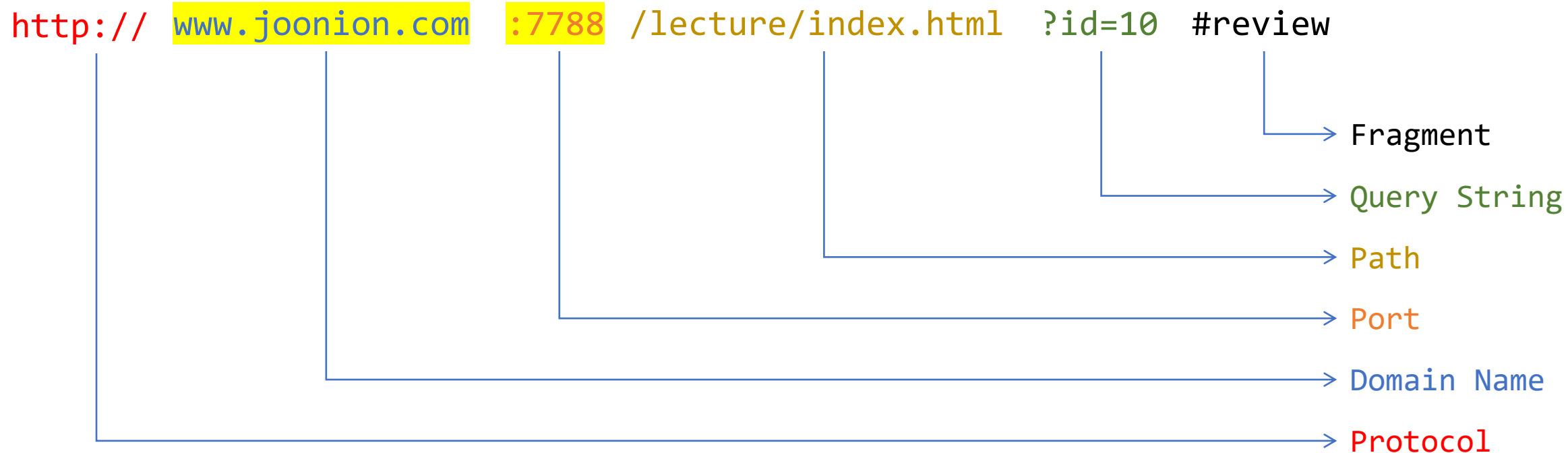






### URL: Uniform Resource Locator

월드와이드웹에서 웹 페이지나 파일의 위치(주소)를 나타내는 문자열  
웹 브라우저가 웹 서버에게 요청을 보낼 때 사용하는 주소

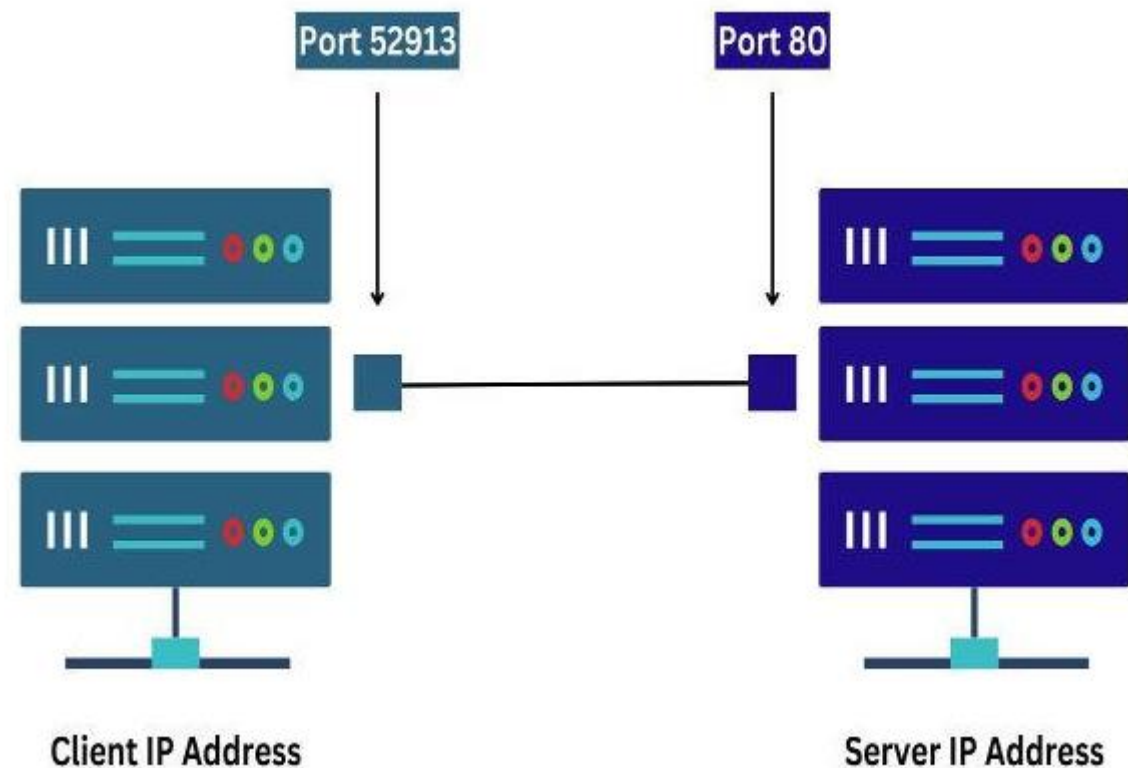




### 포트 번호: *Port* Number

하나의 컴퓨터에서 여러 네트워크 서비스가 통신할 수 있으므로 포트 번호로 구분

- 이메일 통신 (SMTP): 25
- 도메인 네임 서비스 (DNS): 53
- 웹 서버 ([HTTP](#)): 80
- 보안 웹 서버 (HTTPS): 443
- *FastAPI* 서비스: 8000
- *Streamlit* 서비스: 8501





### HTML과 JSON

HTML: *HyperText Markup Language*

사람이 읽기 쉬운 웹 페이지를 만들 수 있도록 정의한 마크업 언어

JSON: *JavaScript Object Notation*

사람 뿐만 아니라 프로그램도 쉽게 읽고 처리 가능한 텍스트 형식



### XML

```
<empinfo>
  <employees>
    <employee>
      <name>James Kirk</name>
      <age>40</age>
    </employee>
    <employee>
      <name>Jean-Luc Picard</name>
      <age>45</age>
    </employee>
    <employee>
      <name>Wesley Crusher</name>
      <age>27</age>
    </employee>
  </employees>
</empinfo>
```

### JSON

```
{ "empinfo" :
  {
    "employees" : [
      {
        "name" : "James Kirk",
        "age" : 40,
      },
      {
        "name" : "Jean-Luc Picard",
        "age" : 45,
      },
      {
        "name" : "Wesley Crusher",
        "age" : 27,
      }
    ]
  }
}
```



### Hello, Flask!

```
# app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello, Flask!"
```

> flask run

--app 옵션이 없으면 디폴트로 `app.py`를 실행함.

> curl http://127.0.0.1:5000



### URL 변수

```
# ex01.py
from flask import Flask
app = Flask(__name__)

@app.route('/user/<username>')
def show_user_profile(username):
    return f"User: {username}"
```

```
> flask --app ex01 run --debug -reload
```

```
http://127.0.0.1:5000/user/joonion
```



### HTTP 메서드

```
# ex02.py
from flask import Flask, request
app = Flask(__name__)

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "GET":
        return "GET request"
    else:
        return "POST request"
```

```
> curl -X POST http://127.0.0.1:5000/login
```

```
> curl -X GET http://127.0.0.1:5000/login
```



### HTTP 요청 처리

```
# ex03.py
from flask import Flask, request
app = Flask(__name__)

@app.route("/query")
def query_example():
    param = request.args.get("param")
    return f"Requested param: {param}"
```

```
> curl -X GET http://127.0.0.1:5000/query?param=hello
Requested param: hello
```





### HTTP 응답 처리 (JSON)

```
# ex04.py
from flask import Flask, jsonify
app = Flask(__name__)

@app.route("/json")
def json_example():
    return jsonify({"message": "Hello, Flask!"})
```

```
> curl -X GET http://127.0.0.1:5000/json
{
  "message": "Hello, Flask!"
}
```



### 템플릿 사용

```
# ex05.py
from flask import Flask, render_template
app = Flask(__name__)

@app.route("/hello/<name>")
def hello_name(name):
    return render_template("hello.html", name=name)
```

```
> curl http://127.0.0.1:5000/hello/joonion
```

```
raise TemplateNotFound(template)
jinja2.exceptions.TemplateNotFound: hello.html
```

서버에 templates 폴더가 없기 때문에 발생하는 오류



templates 폴더에 index.html로 저장

```
<html>
  <head>
    <title>Hello</title>
  </head>
  <body>
    <h1>Hello, {{name}}!</h1>
  </body>
</html>
```

```
> curl http://127.0.0.1:5000/hello/joonion
```

```
<html>
  <head>
    <title>Hello</title>
  </head>
  <body>
    <h1>Hello, joonion!</h1>
  </body>
</html>
```

**Hello, joonion!**



### 반복문과 매크로의 사용

```
# ex06.py
from flask import Flask, render_template
app = Flask(__name__)

@app.route("/messages/<name>")
def show_messages(name):
    subjects = ["전이 학습 기반 모델", "웹 기반 AI 서비스", "클라우드 기반 AI 서비스"]
    return render_template("messages.html", name=name, subjects=subjects)

<!-- templates/macros.html -->
{% macro display_message(message) %}
<p> {{message}} </p>
{% endmacro %}
```



```
<!-- /templates/messages.html -->
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>매크로를 사용하는 예제</title>
</head>
<body>
    {% from "macros.html" import display_message %}
    <h1>Hello, {{name}}!</h1>
    {{ display_message('KDT 교육과정에 오신 것을 환영합니다.') }}
    {{ display_message('이 교육과정에서 이런 것을 배웁니다.') }}
    <ul>
        {% for subject in subjects %}
            <li>{{ subject }}</li>
        {% endfor %}
    </ul>
    {{ display_message('뜨거운 여름, 한 번 불태워봅시다!') }}
</body>
</html>
```



```
> curl http://127.0.0.1:5000/messages/주니온
```

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>매크로를 사용하는 예제</title>
</head>
<body>
  <h1>Hello, 주니온!</h1>
  <p> KDT 교육과정에 오신 것을 환영합니다. </p>
  <p> 이 교육과정에서 이런 것을 배웁니다. </p>
  <ul>
    <li>전이 학습 기반 모델</li>
    <li>웹 기반 AI 서비스</li>
    <li>클라우드 기반 AI 서비스</li>
  </ul>
  <p> 뜨거운 여름, 한 번 불태워봅시다! </p>
</body>
</html>
```

# Hello, 주니온!

KDT 교육과정에 오신 것을 환영합니다.

이 교육과정에서 이런 것을 배웁니다.

- 전이 학습 기반 모델
- 웹 기반 AI 서비스
- 클라우드 기반 AI 서비스

뜨거운 여름, 한 번 불태워봅시다!



### 정적 파일(이미지) 다루기

```
# ex07.py
from flask import Flask, send_from_directory
app = Flask(__name__)

@app.route("/image")
def get_image():
    return send_from_directory('static', 'joonion.png')
```

```
> curl http://127.0.0.1:5000/image --output joonion.png
```





### 세션 관리

```
# ex08.py
from flask import Flask, session
app = Flask(__name__)
app.secret_key = "my secret key"

@app.route("/set_session/<name>")
def set_session(name):
    session["username"] = name
    return "세션에 사용자 이름이 설정되었습니다."

@app.route("/get_session")
def get_session():
    username = session.get("username")
    if username:
        return f"사용자 이름: {username}"
    else:
        return f"현재 세션이 설정되지 않았습니다."
```





```
> curl http://127.0.0.1:5000/get_session
```

현재 세션이 설정되지 않았습니다.

```
> curl -c cookies.txt http://127.0.0.1:5000/set_session/joonion
```

세션에 사용자 이름이 설정되었습니다.

```
> curl -b cookies.txt http://127.0.0.1:5000/get_session
```

사용자 이름: joonion

Flask의 session은 서버 메모리가 아니라 브라우저 쿠키에 저장되는 signed cookie 기반 세션

- 세션은 서버가 알아서 사용자를 기억하는 것이 아니라, 클라이언트가 쿠키를 계속 보내주기 때문에 유지됨
- 웹 브라우저는 이 과정을 자동으로 해주고, curl은 명시적으로 쿠키를 저장하고 다시 보내야 함.



### 간단한 Question-Answering 서비스 만들기

#### 실습 목표:

Flask를 이용해서 간단한 Question-Answer 웹 서비스를 만들어본다.

- 사용자가 웹 브라우저에서 질문을 입력하면 이에 대한 답변을 생성하는 웹 서비스
- HTML form 태그를 이용하여 POST로 요청
- Hugging Face 파이프라인(text-generation)을 이용해서 답변을 생성
- 사용할 모델: Qwen/Qwen2.5-0.5B-Instruct

#### 프로젝트 구조:

```
qna/  
├─ app.py  
└─ templates/  
    ├─ index.html  
    └─ answer.html
```



다음과 같은 컨텍스트를 제공하고,

- 경북대학교에서 운영하는 KDT14기 교육과정은 경일대학교에서 강의를 진행합니다.
- 주니온 교수는 전이학습 기반 모델, 웹 기반, 클라우드 기반 AI 서비스 과목을 강의합니다.

다음과 같은 질문에 답변을 생성한다.

- KDT14기 교육 강사는?
- KDT14기 교육 과목은?
- 주니온 교수가 강의하는 과목은?
- 주니온 교수가 강의하는 장소는?
- 컴퓨터 과학의 아버지는?



### 플라스크 메인 파일: app.py

```
# qna/app.py
from dotenv import load_dotenv
from huggingface_hub import login, whoami

load_dotenv()
login()
print(f"Hugging Face Login: {whoami()['name']}")

from transformers import pipeline
qna = pipeline(
    task="text-generation",
    model="Qwen/Qwen2.5-0.5B-Instruct",
    max_new_tokens=100,
    do_sample=False,
    return_full_text=False,
)
print(f"Pipeline Model: {qna.model.name_or_path}")
```



```
context = """
```

```
경북대학교에서 운영하는 KDT14기 교육과정은 경일대학교에서 강의를 진행합니다.  
주니온 교수는 전이학습 기반 모델, 웹 기반, 클라우드 기반 AI 서비스 과목을 강의합니다.
```

```
"""
```

```
print(f"Context: {context}")
```

```
from flask import Flask, request, render_template
```

```
app = Flask(__name__)
```

```
app.secret_key = "my secret key"
```

```
@app.route("/")
```

```
def index():
```

```
    return render_template("index.html")
```



```
@app.route("/ask", methods=["POST"])
def ask():
    question = request.form.get("question")

    if not question:
        return render_template("answer.html",
                                question=question,
                                answer="질문을 입력하세요."
                                )

    prompt = [
        {"role": "system", "content": "문맥만 이용하여 한 문장으로 답하세요."},
        {"role": "user", "content": f"문맥: {context}, 질문: {question}"}
    ]
```



```
result = qna(prompt)

return render_template(
    "answer.html",
    question=question,
    answer=result[0]["generated_text"].strip()
)
```



### templates/index.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>무엇이든 물어보살!</title>
  <style>
    body { text-align: center; background-color: #f5f5f5; }
    input { width: 300px; padding: 10px; font-size: 16px; }
    button { padding: 10px; font-size: 16px; cursor: pointer; }
  </style>
</head>
<body>
<h1>무엇이든 물어보살!</h1>
<form action="/ask" method="post">
  <input type="text" name="question" placeholder="질문을 입력하세요">
  <button type="submit">질문하기</button>
</form>
</body>
```





### templates/answer.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>무엇이든 물어보살!</title>
  <style>
    body { text-align: center; background-color: #f5f5f5; }
    p { width: 600px; margin: 10px auto; padding: 10px; text-align: left; }
  </style>
</head>
<body>
  <h1>응답 결과</h1>
  <p><b>질문:</b> {{question}}</p>
  <p><b>답변:</b> {{answer}}</p>
  <a href="/">다시 질문하기</a>
</body>
```



### 무엇이든 물어보살!

질문하기

### 응답 결과

**질문:** 주니온 교수가 강의하는 과목은?

**답변:** 주니온 교수는 전이학습 기반 모델과 웹 기반, 클라우드 기반 AI 서비스 과목을 강의합니다.

[다시 질문하기](#)



### Exercise 1.1:

최근 질문 내용을 기억할 수 있도록, 사용자의 질문과 모델의 답변을 session에 저장하시오.

프로젝트 구조:

```
qna2/  
├─ app.py  
└─ templates/  
    ├─ index.html  
    └─ answer.html
```



app.py의 ask() 함수에서 다음과 같이 질문과 답변을 session에 저장하도록 수정.

```
result = qna(prompt)

chat = {
    "question": question,
    "answer": result[0]["generated_text"].strip(),
}
session["chat"] = chat
print(session["chat"])

return render_template(
    "answer.html",
    question=question,
    answer=result[0]["generated_text"].strip()
)
```



app.py의 index() 함수에서 session 정보를 index.html에 넘겨주도록 수정.

```
from flask import Flask, request, render_template, session

@app.route("/")
def index():
    chat = session.get('chat')
    if chat:
        return render_template("index.html", chat=chat)
    else:
        return render_template("index.html", chat=None)
```



index.html에서 최근에 session에 저장된 질문과 답변을 HTML로 출력하도록 수정.

```
<body>
<h1>무엇이든 물어보살!</h1>
{% if chat %}
    <p><b>질문:</b> {{chat['question']}}</p>
    <p><b>답변:</b> {{chat['answer']}}</p>
{% endif %}
<form action="/ask" method="post">
    <input type="text" name="question" placeholder="질문을 입력하세요">
    <button type="submit">질문하기</button>
</form>
</body>
```



### 무엇이든 물어보살!

**질문:** 컴퓨터 과학의 아버지는?

**답변:** 컴퓨터 과학의 아버지는 주니온 교수입니다.



### Exercise 1.2:

이전 대화의 모든 내용을 유지할 수 있도록, 사용자의 질문과 모델의 답변을 session에 저장하시오.

프로젝트 구조:

```
qna3/  
├─ app.py  
└─ templates/  
    ├─ index.html  
    └─ answer.html
```





app.py의 ask() 함수에서 질문과 답변을 session에 chat\_history 리스트로 저장하도록 수정.

```
result = qna(prompt)

chat = {
    "question": question,
    "answer": result[0]["generated_text"].strip(),
}

chat_history = session.get("chat_history", [])
chat_history.append(chat)
session["chat_history"] = chat_history
print(session["chat_history"])
```



app.py의 index() 함수에서 chat\_history 정보를 index.html에 넘겨주도록 수정.

```
@app.route("/")
def index():
    chat_history = session.get('chat_history', [])
    if chat_history:
        return render_template("index.html", chat_history=chat_history)
    else:
        return render_template("index.html", chat_history=None)
```



index.html에서 chat\_history에 저장된 질문과 답변을 모두 출력하도록 수정.

```
<body>
<h1>무엇이든 물어보살!</h1>
{% if chat_history %}
    {% for chat in chat_history %}
        <p><b>질문:</b> {{chat['question']}}</p>
        <p><b>답변:</b> {{chat['answer']}}</p>
    {% endfor %}
{% endif %}
<form action="/ask" method="post">
    <input type="text" name="question" placeholder="질문을 입력하세요">
    <button type="submit">질문하기</button>
</form>
</body>
```



### 무엇이든 물어보살!

**질문:** 컴퓨터 과학의 아버지는?

**답변:** 컴퓨터 과학의 아버지는 주니온 교수입니다.

**질문:** 주니온 교수가 강의하는 장소는?

**답변:** 경북대학교

**질문:** KDT14기 교육 장소는?

**답변:** 경북대학교

**질문:** 주니온 교수가 강의하는 과목은?

**답변:** 주니온 교수는 전이학습 기반 모델과 웹 기반, 클라우드 기반 AI 서비스 과목을 강의합니다.

# 경북대학교 AI·BigData 전문가 양성과정

## Lecture 2.

### RESTful APIs and FastAPI Basics





### 현대적 웹 서비스 모델의 진화

초기 웹은 주로 HTML 문서를 반환하는 단순한 구조의 웹 사이트 중심  
하지만 복잡하고 다양한 웹 서비스의 폭발적 증가로 인해 웹 서비스 아키텍처가 진화함.

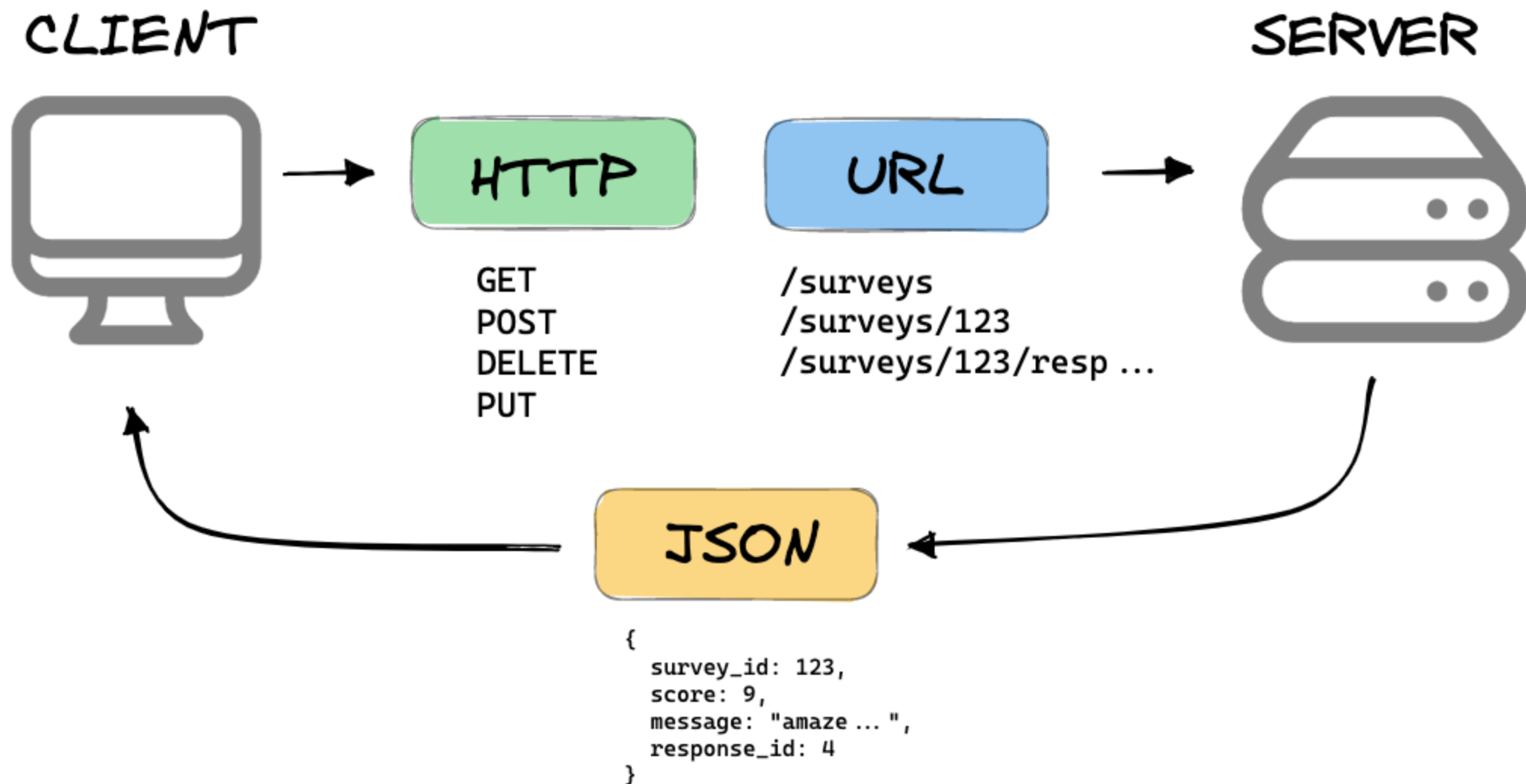
- **REST: RE**presentational **S**tate **T**ransfer

HTTP를 효율적으로 활용하여 자원(resource)을 관리하는 웹 API 설계 규칙

HTTP *Method*는 동사(*action*)로, *URL*은 명사(*resource*)로 정의

- **FastAPI:**

RESTful API를 쉽고 빠르게 개발하기 위해 만들어진 Python 웹 프레임워크





### Hello, FastAPI!

```
# app.py
from fastapi import FastAPI
app = FastAPI()
```

```
@app.get("/")
def hello():
    return {"message": "Hello, FastAPI!"}
```



FastAPI는 자동으로 JSON 직렬화 및 문서화 처리를 하여 응답





```
> uvicorn app:app --reload
```

```
INFO:      Will watch for changes in these directories: ['D:\\knu-kdt\\web\\fastapi']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [24616] using WatchFiles
INFO:      Started server process [17256]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

```
> http localhost:8000/
```

```
HTTP/1.1 200 OK
```

```
content-length: 17
```

```
content-type: application/json
```

```
date: Fri, 26 Jun 2026 00:44:06 GMT
```

```
server: uvicorn
```

```
{
```

```
  "message": "Hello, FastAPI!"
```

```
}
```

```
pretty print 적용 ☒
```

```
{
  "message": "Hello, FastAPI!"
}
```



### 경로 매개변수와 쿼리 매개변수

```
# ex01.py
from fastapi import FastAPI
app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id):
    return {"item_id": item_id}

@app.get("/items/")
def read_items(skip=0, limit=10):
    return {"skip": skip, "limit": limit}
```

```
> uvicorn ex01:app --reload
```



> http localhost:8000/items/5

```
{  
  "item_id": "5"  
}
```

↘ 경로 매개변수

> http localhost:8000/items/

```
{  
  "limit": 10,  
  "skip": 0  
}
```

↘ 쿼리 매개변수

> http "localhost:8000/items/?skip=20&limit=50"

```
{  
  "limit": "50",  
  "skip": "20"  
}
```



### HTTP 메서드

```
# ex02.py
from fastapi import FastAPI
app = FastAPI()
```

```
todos = {}
next_id = 1
```

```
@app.get("/todos")
def get_todos():
    return todos
```



```
@app.post("/todos")
def create_todo(title: str):
    global next_id

    todo = {
        "id": next_id,
        "title": title,
        "completed": False
    }
    todos[next_id] = todo
    next_id += 1

    return todo
```



```
@app.put("/todos/{todo_id}")
def update_todo(todo_id: int, completed: bool):
    if todo_id not in todos:
        return {"error": "Todo not found"}

    todos[todo_id]["completed"] = completed

    return todos[todo_id]

@app.delete("/todos/{todo_id}")
def delete_todo(todo_id: int):
    if todo_id not in todos:
        return {"error": "Todo not found"}

    deleted = todos.pop(todo_id)

    return {
        "message": "Todo deleted",
        "todo": deleted
    }
```



```
> http POST localhost:8000/todos title==여행하기
{
  "completed": false,
  "id": 1,
  "title": "여행하기"
}
```

```
> http PUT localhost:8000/todos/1 completed==true
{
  "completed": true,
  "id": 1,
  "title": "여행하기"
}
```

```
> http DELETE localhost:8000/todos/1
{
  "message": "Todo deleted",
  "todo": {
    "completed": true,
    "id": 1,
    "title": "여행하기"
  }
}
```



### 스트리밍 응답

```
# ex03.py
from fastapi import FastAPI
app = FastAPI()

import time
from fastapi.responses import StreamingResponse

def stream_generator():
    for i in range(10):
        yield f"data chunk {i}: [{time.strftime('%H:%M:%S')}]\\n"
        time.sleep(0.5)

@app.get("/stream")
def get_answer_stream():
    return StreamingResponse(
        content=stream_generator(),
        media_type="text/plain"
    )
```





```
# ex03_client.py
import requests

url = "http://127.0.0.1:8000/stream"
response = requests.get(url, stream=True)

for chunk in response.iter_lines():
    if chunk:
        print(chunk.decode("utf-8"))
```

```
> python ex03_client.py
data chunk 0: [07:25:58]
data chunk 1: [07:25:59]
data chunk 2: [07:25:59]
data chunk 3: [07:26:00]
data chunk 4: [07:26:00]
data chunk 5: [07:26:01]
data chunk 6: [07:26:01]
data chunk 7: [07:26:02]
data chunk 8: [07:26:02]
data chunk 9: [07:26:03]
```



### 비동기 처리: *Asynchronous* Processing

```
# ex04.py
import asyncio

async def func1():
    print("func1: Start")
    await asyncio.sleep(2)
    print("func1: End")

async def func2():
    print("func2: Start")
    await asyncio.sleep(1)
    print("func2: End")

async def main():
    await asyncio.gather(func1(), func2())

asyncio.run(main())
```



### `async def:`

비동기 함수의 선언. 이 함수 내에서만 `await` 키워드를 사용할 수 있다.

비동기 함수로 선언된 함수는 호출할 경우 즉시 `Future` 객체를 반환한다.

이 함수는 다른 코드와 **동시에** 실행될 수 있지만, 별도의 스레드나 프로세스에서 실행되지 않는다.

### `await:`

비동기 함수 내에서만 사용할 수 있는 키워드.

다음에 오는 비동기 연산이 완료될 때까지 현재 비동기 함수의 실행을 일시적으로 중단한다.

하지만 프로그램 전체가 멈추는 것은 아니고, 다른 비동기 함수나 작업이 실행될 수 있다.

비동기 = 기다리는 동안 다른 일을 처리하는 것



```
> python ex04.py  
func1: Start  
func2: Start  
func2: End  
func1: End
```

func1()과 func2()는 동시에 시작되지만,  
func2()는 1초 후에 끝나고, func1()은 2초 후에 끝난다.

func1()은 대기 중일 때도 func2()의 실행을 방해하지 않으므로  
func2: End가 func1: End보다 먼저 출력된다.



### FastAPI에서의 비동기 처리

```
# ex05.py
from fastapi import FastAPI
app = FastAPI()

import asyncio

async def fetch_data():
    await asyncio.sleep(5)
    return {"data": "some_data"}

@app.get("/")
async def read_root():
    data = await fetch_data()
    return {"message": "Hello, World!", "data": data}
```



```
> http localhost:8000/  
HTTP/1.1 200 OK  
content-length: 55  
content-type: application/json  
date: Thu, 02 Jul 2026 23:40:17  
GMTserver: uvicorn
```

```
{  
  "data": {  
    "data": "some_data"  
  },  
  "message": "Hello, World!"  
}
```

```
> http localhost:8000/  
HTTP/1.1 200 OK  
content-length: 55  
content-type: application/json  
date: Thu, 02 Jul 2026 23:40:18  
GMTserver: uvicorn
```

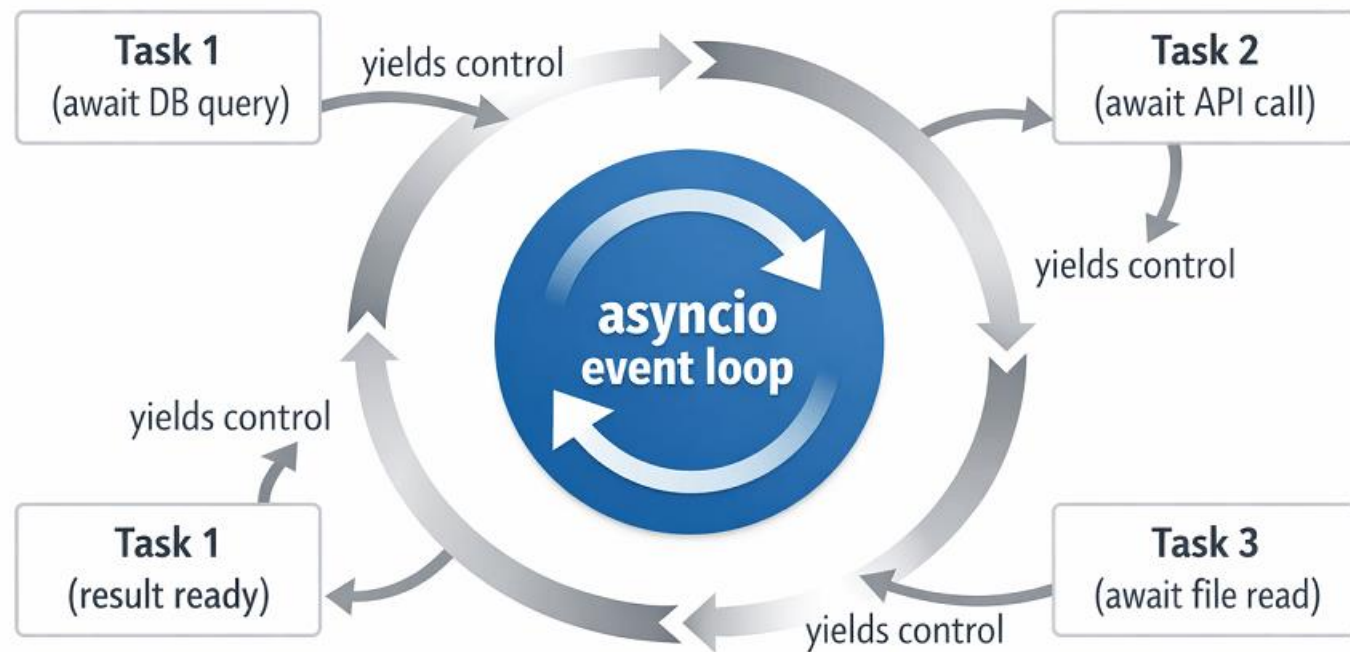
```
{  
  "data": {  
    "data": "some_data"  
  },  
  "message": "Hello, World!"  
}
```



### 이벤트 루프: *Event Loop*

이벤트 루프는 “지금 실행 가능한 작업”을 계속 찾아 실행하는 관리자.

FastAPI는 여러 사용자의 요청을 동시에 처리해야 하므로, 내부적으로 이벤트 루프를 사용한다.





### 동기 처리와 비동기 처리 비교

```
# ex06.py
from fastapi import FastAPI
app = FastAPI()

import time, asyncio

@app.get("/sync")
def read_item_sync():
    start_time = time.time()
    time.sleep(1)
    elapsed_time = time.time() - start_time
    return {"type": "sync/", "elapsed_time": elapsed_time}

@app.get("/async")
async def read_item_async():
    start_time = time.time()
    await asyncio.sleep(1)
    elapsed_time = time.time() - start_time
    return {"type": "async/", "elapsed_time": elapsed_time}
```





```
# ex06_client.py
import time, requests, asyncio, aiohttp

def call_sync(url, times):
    start_time = time.time()
    for _ in range(times):
        requests.get(url)
    elapsed_time = time.time() - start_time
    print(f"[Sync] elapsed_time: {elapsed_time:.2f}")

async def call_async(url, times):
    start_time = time.time()
    async with aiohttp.ClientSession() as session:
        tasks = [session.get(url) for _ in range(times)]
        await asyncio.gather(*tasks)
    elapsed_time = time.time() - start_time
    print(f"[Sync] elapsed_time: {elapsed_time:.2f}")

if __name__ == "__main__":
    api_url = "http://127.0.0.1:8000/"
    call_sync(api_url + "sync", 10)
    asyncio.run(call_async(api_url + "async", 10))
```

```
> python ex06_client.py
[Sync] elapsed_time: 14.52
[Sync] elapsed_time: 1.01
```



### 파일 업로드

```
# ex07.py
from fastapi import FastAPI
app = FastAPI()

from fastapi import File, UploadFile
from pathlib import Path
import shutil

UPLOAD_DIR = Path("uploads")
UPLOAD_DIR.mkdir(exist_ok=True)

@app.post("/uploadfile/")
async def upload_file(file: UploadFile = File(...)):
    file_path = UPLOAD_DIR / file.filename
    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    return {"message": f"파일 업로드 완료: {file_path}"}
```



### 자동 문서화: *Automatic API Documentation*

FastAPI는 API 문서를 자동으로 생성해 주므로 별도의 개발 문서를 작성할 필요가 없다.

- API 목록
- 요청(Request) 형식
- 응답(Response) 형식
- 파라미터 설명
- 데이터 타입

FastAPI의 두 가지 문서 유형:

- Swagger UI: [API 테스트](#) 기능 포함.  
<http://localhost:8000/docs> 에서 확인 가능.
- ReDoc: 가독성이 높지만 테스트 기능이 없다.  
<http://localhost:8000/redoc> 에서 확인 가능.



## FastAPI 0.1.0 OAS 3.1

/openapi.json

### default



POST

/uploadfile/ Upload File



### Schemas



Body\_upload\_file\_uploadfile\_\_post > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object



## FastAPI 0.1.0 OAS 3.1

/openapi.json

### default

**POST** /uploadfile/ Upload File

#### Parameters

Cancel

Reset

No parameters

**Request body** required

multipart/form-data

**file** \* required

string

파일 선택 kdt.pdf

Execute



## Responses

### Curl

```
curl -X 'POST' \  
  'http://127.0.0.1:8000/uploadfile/' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: multipart/form-data' \  
  -F 'file=@kdt.pdf;type=application/pdf'
```

### Request URL

```
http://127.0.0.1:8000/uploadfile/
```

### Server response

#### Code

#### Details

200

#### Response body

```
{  
  "message": "파일 업로드 완료: uploads\\kdt.pdf"  
}
```



Download

#### Response headers

```
content-length: 55  
content-type: application/json  
date: Fri, 03 Jul 2026 01:30:07 GMT  
server: uvicorn
```

## Responses



### Exercise 2.1:

Flask로 구현했던 Question-Answering 서비스를 FastAPI 기반 API 서비스로 구현하시오.

- FastAPI 라우팅
- POST 요청 처리
- Form 데이터 수신
- Hugging Face LLM 호출
- JSON 응답 변환
- HTTPie 또는 Swagger UI를 이용한 API 테스트



프로젝트 구조:

```
qna_fastapi/  
├── api.py      # FastAPI 서버를 작성  
└── rag.py      # LLM 모델을 이용한 질문/답변 함수 작성
```

API 테스트:

```
http -f POST http://localhost:8000/ask question="KDT14기 교육 장소는?"
```



# 경북대학교 AI·BigData 전문가 양성과정

## Lecture 3.

### Frontend UI/UX and Streamlit Basics





### 프론트엔드의 역할

- **Backend:** *RESTfun APIs*

클라이언트의 요청에 대해 데이터를 처리하고 결과를 반환하는 역할

일반적으로 사용자에게 직접 보이지 않는 영역에서 비즈니스 로직을 처리

- **Frontend:** *User Interface / User eXperience*

사용자의 입력을 백엔드에 전달하고, 서버가 반환한 결과를 사용자에게 보여주는 기술

전통적인 웹 UI 개발: HTML, CSS, JavaScript, etc.

**Streamlit:** Python 만으로 웹 UI를 쉽고 빠르게 개발할 수 있는 웹 UI 프레임워크



### Hello, Streamlit!

```
# app.py
import streamlit as st

st.title("Hello, Streamlit!")
```

```
> streamlit run app.py
```

```
2026-06-26 21:43:49.430 Uvicorn server started on 0.0.0.0:8501
```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>

Network URL: <http://155.230.35.169:8501>

**Hello, Streamlit!** ↪



### 텍스트 출력

```
# ex01.py
import streamlit as st

st.title("Streamlit 텍스트 출력 실습")
st.write("이번 실습에서는 Streamlit에서 텍스트를 출력하는 다양한 방법을 배웁니다.")
st.divider()

st.header("1. 일반 텍스트 출력")
st.text("st.text()는 가장 단순한 텍스트 출력 함수입니다.")
st.write("st.write()는 문자열, 숫자, 데이터프레임 등 다양한 객체를 출력할 수 있습니다.")
st.header("2. Markdown 출력")
st.markdown("""
Streamlit에서는 **Markdown 문법**을 사용할 수 있습니다.
예를 들어, 다음과 같은 표현이 가능합니다.
- **굵은 글씨**
- *기울임 글씨*
- `코드 표현`
- [Streamlit 공식 문서 링크](https://docs.streamlit.io/)
""")
```



```
st.header("3. 코드와 수식 출력")
```

```
code = """
```

```
import streamlit as st
```

```
st.title("Hello, Streamlit!")
```

```
"""
```

```
st.code(code, language="python")
```

```
st.write("다음은 소프트맥스 함수입니다.")
```

```
st.latex(r"""
```

```
    p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}
```

```
""")
```

```
st.divider()
```

```
st.success("Streamlit을 이용하면 간단한 웹 페이지를 빠르게 만들 수 있습니다.")
```



# Streamlit 텍스트 출력 실습

이번 실습에서는 Streamlit에서 텍스트를 출력하는 다양한 방법을 배웁니다.

---

## 1. 일반 텍스트 출력

`st.text()`는 가장 단순한 텍스트 출력 함수입니다.

`st.write()`는 문자열, 숫자, 데이터프레임 등 다양한 객체를 출력할 수 있습니다.

## 2. Markdown 출력

Streamlit에서는 **Markdown 문법**을 사용할 수 있습니다. 예를 들어, 다음과 같은 표현이 가능합니다.

- 굵은 글씨
- 기울임 글씨
- 코드 표현
- [Streamlit 공식 문서 링크](#)



## 3. 코드와 수식 출력

```
import streamlit as st
st.title("Hello, Streamlit!")
```

다음은 소프트맥스 함수입니다.

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Streamlit을 이용하면 간단한 웹 페이지를 빠르게 만들 수 있습니다.



### 데이터 출력과 차트 그리기

```
# ex02.py
import streamlit as st
import seaborn as sns

penguins = sns.load_dataset("penguins")

st.title("팔머 펭귄 데이터셋")

st.header("1. 데이터 프레임")
st.dataframe(penguins)

st.header("2. 부리의 길이-깊이 산점도")
st.write("팔머 펭귄의 부리 길이와 부리 깊이는 상관이 높은가?")
st.scatter_chart(
    data = penguins,
    x="bill_length_mm",
    y="bill_depth_mm",
)
```





## 팔머 펭귄 데이터셋

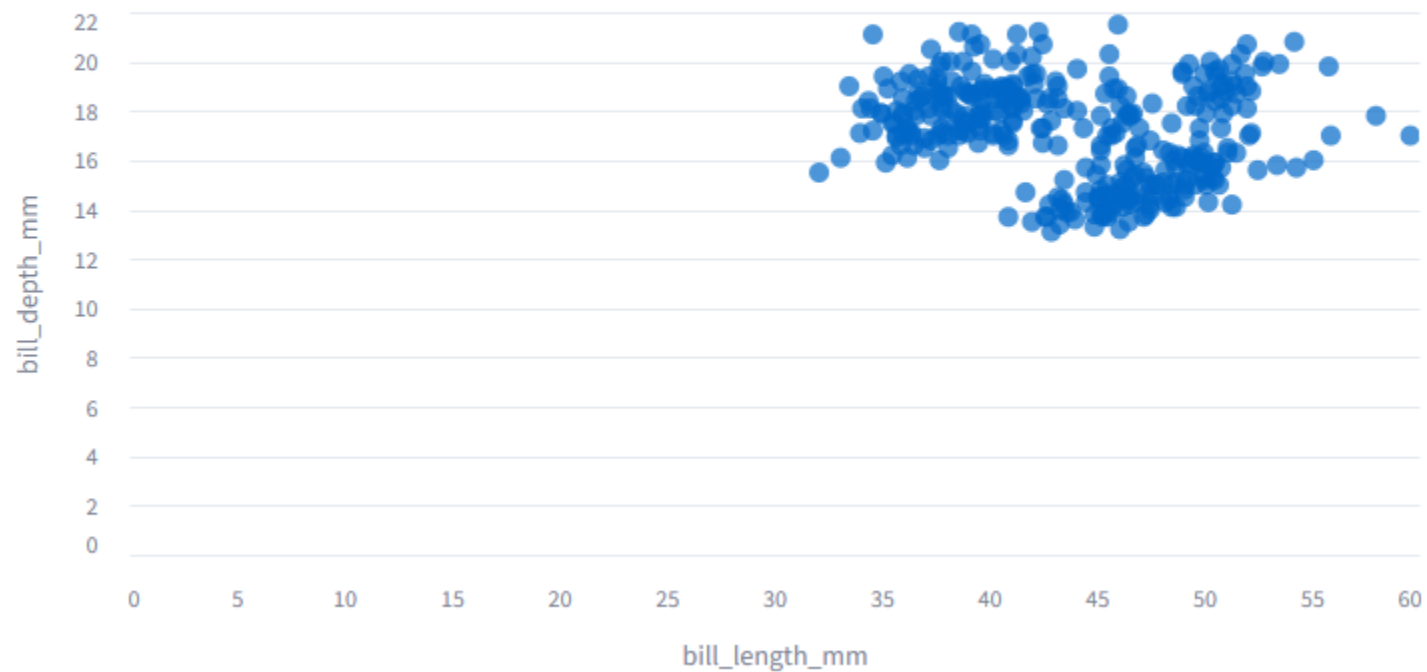
### 1. 데이터 프레임

|   | species | island    | bill_length_mm | bill_depth_mm | flipper_length_mm | body_mass_g | sex    |
|---|---------|-----------|----------------|---------------|-------------------|-------------|--------|
| 0 | Adelie  | Torgersen | 39.1           | 18.7          | 181               | 3750        | Male   |
| 1 | Adelie  | Torgersen | 39.5           | 17.4          | 186               | 3800        | Female |
| 2 | Adelie  | Torgersen | 40.3           | 18            | 195               | 3250        | Female |
| 3 | Adelie  | Torgersen | None           | None          | None              | None        | None   |
| 4 | Adelie  | Torgersen | 36.7           | 19.3          | 193               | 3450        | Female |
| 5 | Adelie  | Torgersen | 39.3           | 20.6          | 190               | 3650        | Male   |
| 6 | Adelie  | Torgersen | 38.9           | 17.8          | 181               | 3625        | Female |
| 7 | Adelie  | Torgersen | 39.2           | 19.6          | 195               | 4675        | Male   |
| 8 | Adelie  | Torgersen | 34.1           | 18.1          | 193               | 3475        | None   |
| 9 | Adelie  | Torgersen | 42             | 20.2          | 190               | 4250        | None   |



## 2. 부리의 길이-깊이 산점도

팔머 펭귄의 부리 길이와 부리 깊이는 상관관계가 높은가?





### Layouts: 탭 활용

```
# ex03.py
import streamlit as st
import seaborn as sns
import altair as alt

st.title("Penguins 데이터셋 시각화")

penguins = sns.load_dataset("penguins")

tab1, tab2, tab3 = st.tabs(["데이터 보기", "산점도 보기", "종별 시각화"])

with tab1:
    st.header("Penguins 데이터셋")
    st.dataframe(penguins)
```



```
with tab2:
    st.header("부리 길이와 부리 깊이의 관계")

    chart = alt.Chart(penguins).mark_circle(size=60).encode(
        x=alt.X(
            "bill_length_mm:Q",
            scale=alt.Scale(zero=False),
            title="Bill length (mm)"
        ),
        y=alt.Y(
            "bill_depth_mm:Q",
            scale=alt.Scale(zero=False),
            title="Bill depth (mm)"
        ),
    ).interactive()

    st.altair_chart(chart, width='stretch')
```



```
with tab3:
    st.header("펭귄의 종별 시각화")

    chart = alt.Chart(penguins).mark_circle(size=60).encode(
        x=alt.X(
            "bill_length_mm:Q",
            scale=alt.Scale(zero=False),
            title="Bill length (mm)"
        ),
        y=alt.Y(
            "bill_depth_mm:Q",
            scale=alt.Scale(zero=False),
            title="Bill depth (mm)"
        ),
        color="species:N",
        tooltip=["species", "bill_length_mm", "bill_depth_mm"]
    ).interactive()

    st.altair_chart(chart, width='stretch')
```



# Penguins 데이터셋 시각화

[데이터 보기](#) [산점도 보기](#) [종별 시각화](#)

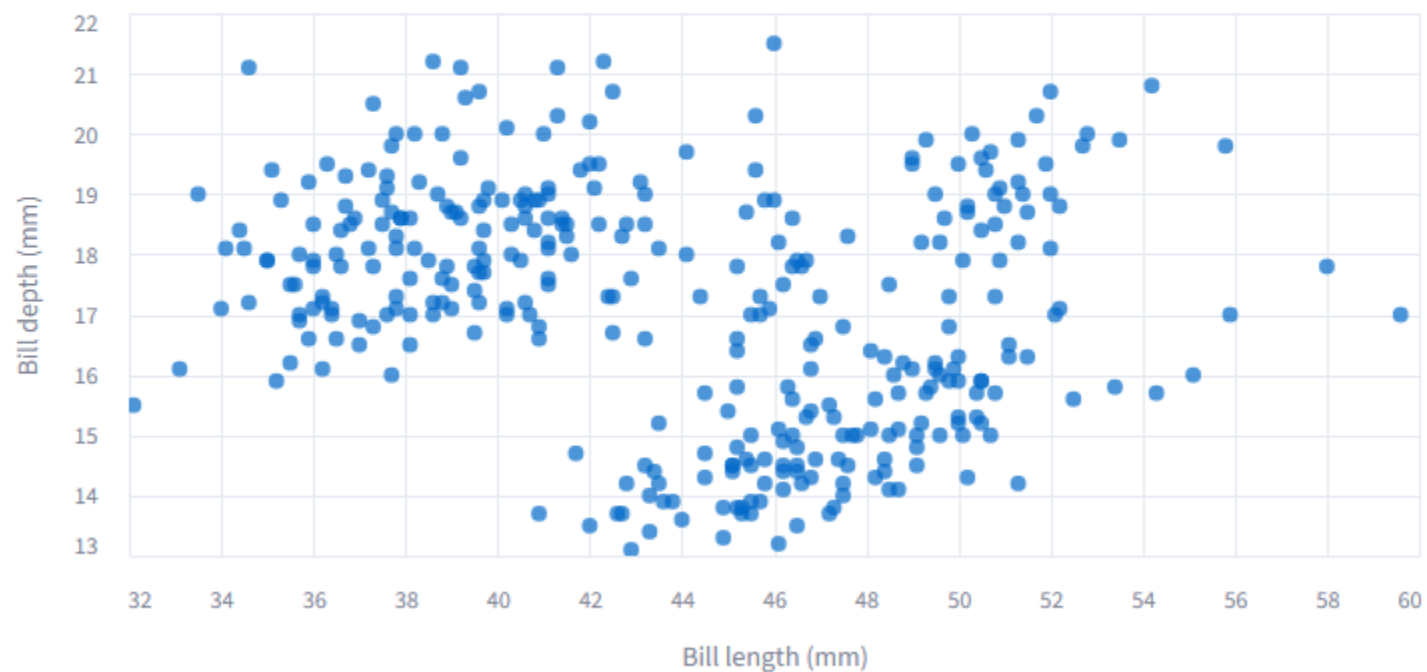
## Penguins 데이터셋

|   | species | island    | bill_length_mm | bill_depth_mm | flipper_length_mm | body_mass_g | sex    |
|---|---------|-----------|----------------|---------------|-------------------|-------------|--------|
| 0 | Adelie  | Torgersen | 39.1           | 18.7          | 181               | 3750        | Male   |
| 1 | Adelie  | Torgersen | 39.5           | 17.4          | 186               | 3800        | Female |
| 2 | Adelie  | Torgersen | 40.3           | 18            | 195               | 3250        | Female |
| 3 | Adelie  | Torgersen | None           | None          | None              | None        | None   |
| 4 | Adelie  | Torgersen | 36.7           | 19.3          | 193               | 3450        | Female |
| 5 | Adelie  | Torgersen | 39.3           | 20.6          | 190               | 3650        | Male   |
| 6 | Adelie  | Torgersen | 38.9           | 17.8          | 181               | 3625        | Female |
| 7 | Adelie  | Torgersen | 39.2           | 19.6          | 195               | 4675        | Male   |
| 8 | Adelie  | Torgersen | 34.1           | 18.1          | 193               | 3475        | None   |
| 9 | Adelie  | Torgersen | 42             | 20.2          | 190               | 4250        | None   |
|   |         |           |                |               |                   |             |        |



데이터 보기 산점도 보기 종별 시각화

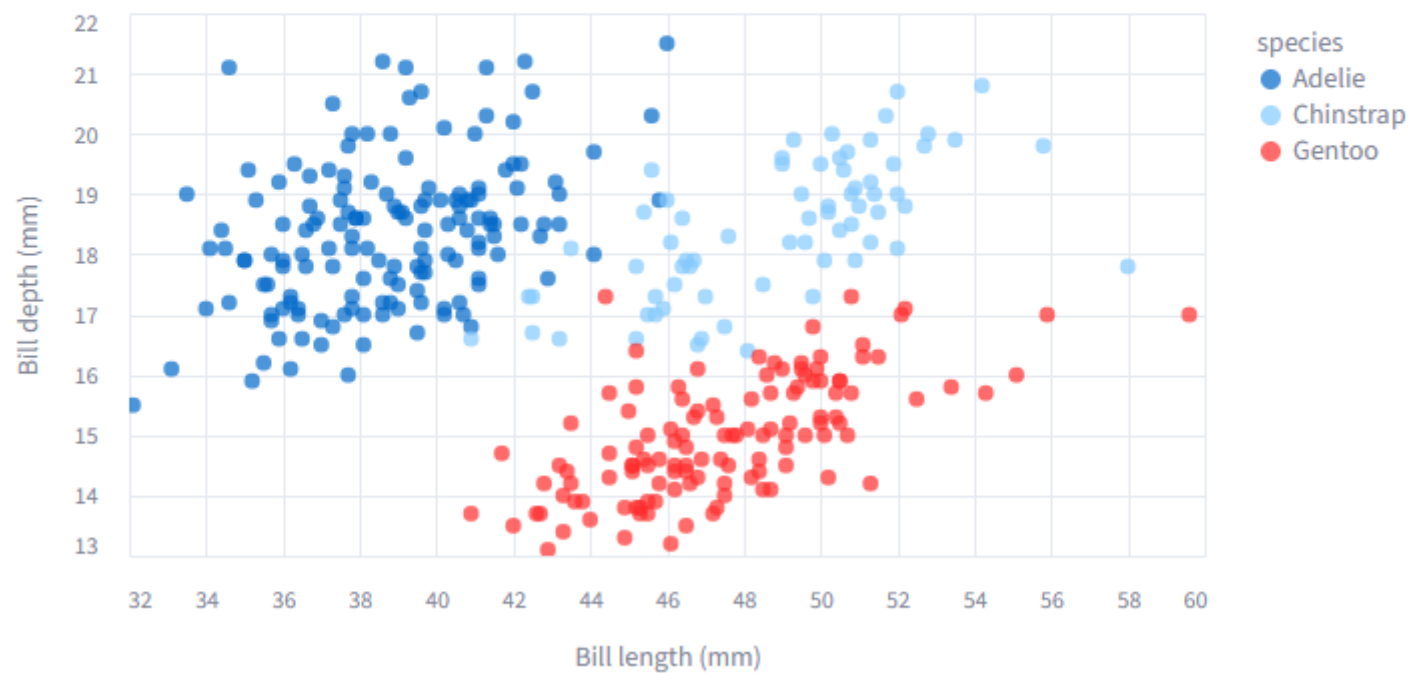
## 부리 길이와 부리 깊이의 관계





데이터 보기 산점도 보기 **종별 시각화**

## 펭귄의 종별 시각화







### Widgets: 입력 위젯 활용

```
# ex04.py
import streamlit as st
import seaborn as sns
import altair as alt

st.title("입력 위젯을 이용한 Penguins 데이터 탐색")

penguins = sns.load_dataset("penguins")
penguins = penguins.dropna()

st.header("1. 원본 데이터")
st.dataframe(penguins)
```



```
st.header("2. 입력 위젯으로 데이터 필터링")

species_list = penguins["species"].unique().tolist()
island_list = penguins["island"].unique().tolist()

selected_species = st.selectbox(
    "펭귄 종을 선택하세요.",
    species_list
)

selected_island = st.multiselect(
    "섬을 선택하세요.",
    island_list,
    default=island_list
)
```



```
min_body_mass, max_body_mass = st.slider(
    "몸무게 범위를 선택하세요.",
    min_value=int(penguins["body_mass_g"].min()),
    max_value=int(penguins["body_mass_g"].max()),
    value=(
        int(penguins["body_mass_g"].min()),
        int(penguins["body_mass_g"].max())
    )
)

show_table = st.checkbox("필터링된 데이터 보기", value=True)

filtered = penguins[
    (penguins["species"] == selected_species) &
    (penguins["island"].isin(selected_island)) &
    (penguins["body_mass_g"] >= min_body_mass) &
    (penguins["body_mass_g"] <= max_body_mass)
]
```



```
st.header("3. 필터링 결과")

st.write(f"선택된 데이터 개수: **{len(filtered)}개**")

if show_table:
    st.dataframe(filtered)
```



```
st.header("4. 산점도")

chart = alt.Chart(filtered).mark_circle(size=70).encode(
    x=alt.X(
        "bill_length_mm:Q",
        scale=alt.Scale(zero=False),
        title="Bill length (mm)"
    ),
    y=alt.Y(
        "bill_depth_mm:Q",
        scale=alt.Scale(zero=False),
        title="Bill depth (mm)"
    ),
    color="species:N",
    tooltip=["species", "bill_length_mm", "bill_depth_mm"]
).interactive()

st.altair_chart(chart, width='stretch')
```



## 입력 위젯을 이용한 Penguins 데이터 탐색

### 1. 원본 데이터

|    | species | island    | bill_length_mm | bill_depth_mm | flipper_length_mm | body_mass_g | sex    |
|----|---------|-----------|----------------|---------------|-------------------|-------------|--------|
| 0  | Adelie  | Torgersen | 39.1           | 18.7          | 181               | 3750        | Male   |
| 1  | Adelie  | Torgersen | 39.5           | 17.4          | 186               | 3800        | Female |
| 2  | Adelie  | Torgersen | 40.3           | 18            | 195               | 3250        | Female |
| 4  | Adelie  | Torgersen | 36.7           | 19.3          | 193               | 3450        | Female |
| 5  | Adelie  | Torgersen | 39.3           | 20.6          | 190               | 3650        | Male   |
| 6  | Adelie  | Torgersen | 38.9           | 17.8          | 181               | 3625        | Female |
| 7  | Adelie  | Torgersen | 39.2           | 19.6          | 195               | 4675        | Male   |
| 12 | Adelie  | Torgersen | 41.1           | 17.6          | 182               | 3200        | Female |
| 13 | Adelie  | Torgersen | 38.6           | 21.2          | 191               | 3800        | Male   |
| 14 | Adelie  | Torgersen | 34.6           | 21.1          | 198               | 4400        | Male   |
|    |         |           |                |               |                   |             |        |



## 2. 입력 위젯으로 데이터 필터링

펭귄 종을 선택하세요.

Adelie



섬을 선택하세요.

Torgersen x

Biscoe x

Dream x



몸무게 범위를 선택하세요.

2700

6300



☒ 필터링된 데이터 보기



## 3. 필터링 결과

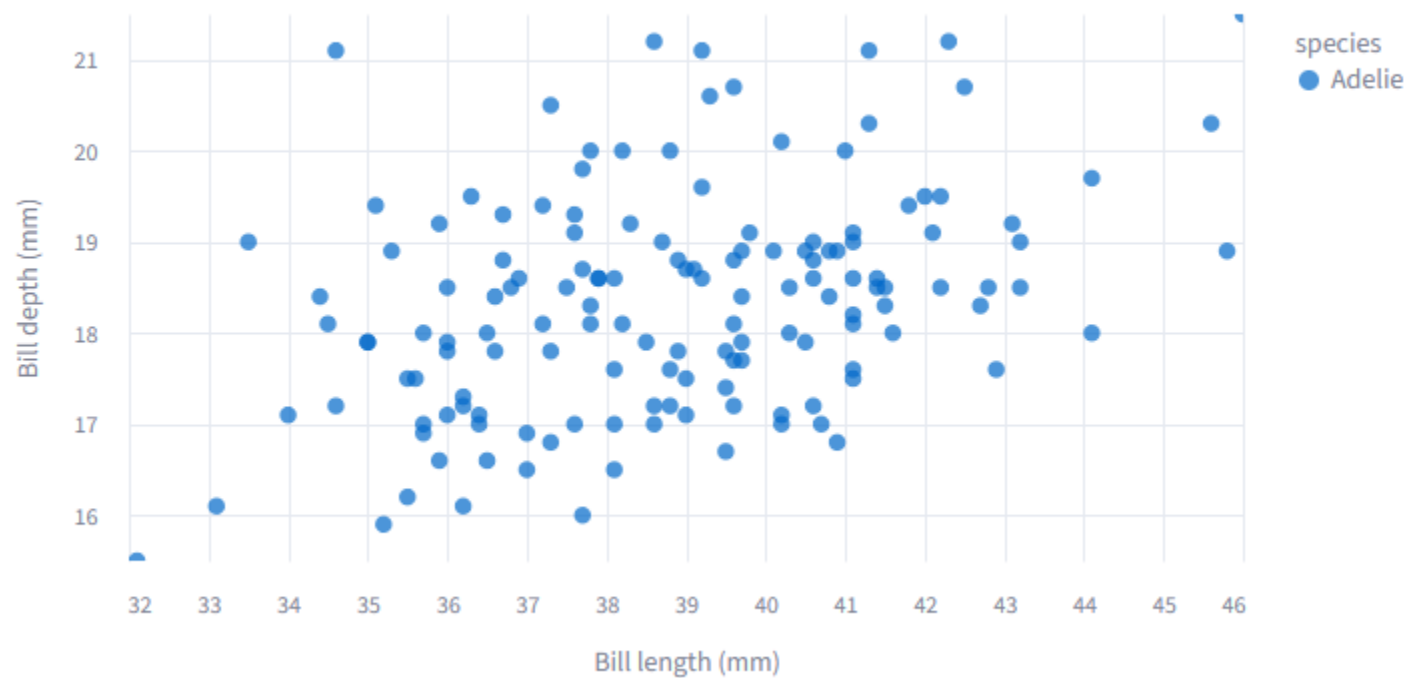
선택된 데이터 개수: 146개

|    | species | island    | bill_length_mm | bill_depth_mm | flipper_length_mm | body_mass_g | sex    |
|----|---------|-----------|----------------|---------------|-------------------|-------------|--------|
| 0  | Adelie  | Torgersen | 39.1           | 18.7          | 181               | 3750        | Male   |
| 1  | Adelie  | Torgersen | 39.5           | 17.4          | 186               | 3800        | Female |
| 2  | Adelie  | Torgersen | 40.3           | 18            | 195               | 3250        | Female |
| 4  | Adelie  | Torgersen | 36.7           | 19.3          | 193               | 3450        | Female |
| 5  | Adelie  | Torgersen | 39.3           | 20.6          | 190               | 3650        | Male   |
| 6  | Adelie  | Torgersen | 38.9           | 17.8          | 181               | 3625        | Female |
| 7  | Adelie  | Torgersen | 39.2           | 19.6          | 195               | 4675        | Male   |
| 12 | Adelie  | Torgersen | 41.1           | 17.6          | 182               | 3200        | Female |
| 13 | Adelie  | Torgersen | 38.6           | 21.2          | 191               | 3800        | Male   |
| 14 | Adelie  | Torgersen | 34.6           | 21.1          | 198               | 4400        | Male   |
|    |         |           |                |               |                   |             |        |





## 4. 산점도





### 세션 상태 관리

```
# ex05.py
import streamlit as st

st.title("세션 상태가 없는 카운터")

count = 0
print(f"카운터가 초기화되었습니다.")

if st.button("Increment"):
    count += 1
    print(count)

st.write("Count =", count)
```

### 세션 상태가 없는 카운터

Increment

Count = 1

버튼을 클릭할 때마다 전체 스크립트가 다시 실행되므로  
버튼을 여러 번 눌러도 항상 1인 상태가 된다.



```
# ex06.py
import streamlit as st

st.title("세션 상태가 있는 카운터")

if "count" not in st.session_state:
    st.session_state.count = 0
    print(f"카운터가 초기화되었습니다.")

if st.button("Increment"):
    st.session_state.count += 1
    print(st.session_state.count)

st.write("Count =", st.session_state.count)
```

### 세션 상태가 있는 카운터

Increment

Count = 15



첫 실행 때만 count가 초기화되고  
이후 rerun에서는 세션 상태의 count 값을 증가함



## 콜백 함수의 사용

```
# ex07.py
import streamlit as st

st.title("콜백을 활용한 카운터")

if "count" not in st.session_state:
    st.session_state.count = 0

def increment_counter():
    st.session_state.count += 1

def decrement_counter():
    st.session_state.count -= 1

st.button("Increment", on_click=increment_counter)
st.button("Decrement", on_click=decrement_counter)

st.write("Count =", st.session_state.count)
```

## 콜백을 활용한 카운터

Increment

Decrement

Count = -8

버튼이 클릭되거나(on\_click)  
위젯의 값이 바뀔 때(on\_change)  
콜백 함수를 호출



```
# ex08.py
import streamlit as st

st.title("입력값이 있는 콜백 함수")

if "count" not in st.session_state:
    st.session_state.count = 0

def increment_counter(value):
    st.session_state.count += value

def decrement_counter(value):
    st.session_state.count -= value

size = st.number_input("얼마나?", value=1, step=1)

st.button("Increment", on_click=increment_counter, args=(size,))
st.button("Decrement", on_click=decrement_counter, args=(size,))

st.write("Count =", st.session_state.count)
```

### 입력값이 있는 콜백 함수

얼마나?

7

Increment

Decrement

Count = -50



### 위젯 상태와 세션 상태의 연결

```
# ex09.py
import streamlit as st

st.title("위젯 상태와 세션 상태")

if "name" not in st.session_state:
    st.session_state.name = ""

st.text_input("이름을 입력하세요", key="name")

st.write("입력한 이름:", st.session_state.name)
st.write("현재 Session State:")
st.write(st.session_state)
```

### 위젯 상태와 세션 상태

이름을 입력하세요

주니온

입력한 이름: 주니온

현재 Session State:

```
{
  "name" : "주니온"
}
```



### Exercise 3.1:

Streamlit 기반 질문과 답변 앱 만들기

- `st.text_input()`을 이용하여 사용자가 질문을 입력할 수 있도록 한다.
- `st.button()`을 누르면 입력한 질문이 `st.session_state`에 저장되도록 한다.
- 저장된 질문 목록을 화면에 순서대로 출력한다.
- 질문이 비어 있으면 저장하지 않고 경고 메시지를 출력한다.
- [전체 삭제] 버튼을 만들고, 누르면 저장된 질문 목록을 모두 삭제한다.
- `st.session_state`를 사용하여 버튼을 눌러도 기존 질문 목록이 유지되도록 한다.

# 경북대학교 AI·BigData 전문가 양성과정

## Lecture 4.

### Building an LLM-based Chat App







knu-kdt/

- |—— ex01.py      # 뇌 없는 챗봇: LLM 없이 **답변** 하기
- |—— ex02.py      # 기억만 할 줄 아는 챗봇: **Session** 정보 관리하기
- |—— ex03.py      # 대화를 할 줄 아는 챗봇: **LLM**의 도입
- |—— ex04.py      # 자연스러운 대화: **Streaming** 도입
- |—— ex05.py      # 지식 기반의 질의 응답: **PDF** 파일 업로드
- |—— ex06.py      # **RAG**: Chroma 벡터 스토어를 활용한 지식 기반 대화



### 뇌 없는 챗봇

```
# ex01.py
import streamlit as st

st.title("무엇이든 물어보살!")

if question := st.chat_input("질문을 입력하세요."):

    with st.chat_message("user"):
        st.markdown(question)

    response = f"와, 너 **핵심**을 *꼭* 짚렀어! {question}이라고 질문하다니!"
    with st.chat_message("assistant"):
        st.markdown(response)
```



### 무엇이든 물어보살! ↔



컴퓨터 과학의 아버지는?



와, 너 **핵심**을 콕 찝렸어! 컴퓨터 과학의 아버지는?이라고 질문하다니!

질문을 입력하세요.





### 기억만 할 줄 아는 챗봇

```
# ex02.py
import streamlit as st

st.title("무엇이든 물어보살!")

if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message['role']):
        st.markdown(message['content'])
```



```
if question := st.chat_input("질문을 입력하세요."):

    with st.chat_message("user"):
        st.markdown(question)
    st.session_state.messages.append({"role": "user", "content": question})

    answer = f"와, 너 **핵심**을 *꼭* 짚렸어!"
    with st.chat_message("assistant"):
        st.markdown(answer)
    st.session_state.messages.append({"role": "assistant", "content": answer})
```



### 무엇이든 물어보살!



컴퓨터 과학의 아버지는?



와, 너 **핵심**을 콕 찔렀어!



컴퓨터 과학의 어머니는?



와, 너 **핵심**을 콕 찔렀어!



싸우자는 거니?



와, 너 **핵심**을 콕 찔렀어!

질문을 입력하세요.





### 대화를 할 줄 아는 챗봇

```
# ex03.py
import streamlit as st
from transformers import pipeline

@st.cache_resource
def load_model():
    return pipeline(
        task="text-generation",
        model="Qwen/Qwen2.5-0.5B-Instruct",
        max_new_tokens=100,
        do_sample=False,
        return_full_text=False,
    )

chatbot = load_model()
```



```
def get_ai_answer(messages):  
    prompt = [  
        {  
            "role": "system",  
            "content": "답변은 한국어로 한 문장으로 답하세요.",  
        },  
        {  
            "role": "user",  
            "content": messages[-1]['content'],  
        },  
    ]  
  
    result = chatbot(prompt)  
    return result[0]["generated_text"]
```





LLM이 질문에 대한 답변을 생성하도록 수정

```
if question := st.chat_input("질문을 입력하세요."):

    with st.chat_message("user"):
        st.markdown(question)
    st.session_state.messages.append({"role": "user", "content": question})

    answer = get_ai_answer(st.session_state.messages)
    with st.chat_message("assistant"):
        st.markdown(answer)
    st.session_state.messages.append({"role": "assistant", "content": answer})
```



### 무엇이든 물어보살!



컴퓨터 과학의 아버지는?



컴퓨터 과학의 아버지는 브래드마인입니다.



주니온 교수가 강의하는 과목은?



주니온 교수가 강의하는 과목은 컴퓨터 프로그래밍입니다.



세종 대왕이 맥북을 던진 사건은 언제 일어났지?



세종대왕이 맥북을 던진 사건은 2018년 3월 15일에 발생했습니다.

질문을 입력하세요.





### 자연스러운 대화를 할 줄 아는 챗봇: 스트리밍 적용

```
# ex04.py
import streamlit as st
from transformers import AutoTokenizer, AutoModelForCausalLM, TextIteratorStreamer
from threading import Thread

@st.cache_resource
def load_model():
    model_id = "Qwen/Qwen2.5-0.5B-Instruct"

    tokenizer = AutoTokenizer.from_pretrained(model_id)
    model = AutoModelForCausalLM.from_pretrained(model_id)

    return tokenizer, model

tokenizer, model = load_model()
```



```
def get_ai_answer_stream(messages):  
    prompt = [  
        {"role": "system", "content": "답변은 한국어로 한 문장으로 답하세요."},  
        {"role": "user", "content": messages[-1]["content"]},  
    ]  
  
    inputs = tokenizer.apply_chat_template(  
        prompt,  
        add_generation_prompt=True,  
        return_tensors="pt",  
        return_dict=True,  
    )  
  
    streamer = TextIteratorStreamer(  
        tokenizer,  
        skip_prompt=True,  
        skip_special_tokens=True,  
    )
```



```
generation_kwargs = {
    **inputs,
    "streamer": streamer,
    "max_new_tokens": 100,
    "do_sample": False,
    "pad_token_id": tokenizer.eos_token_id,
}

thread = Thread(
    target=model.generate,
    kwargs=generation_kwargs,
    daemon=True,
)
thread.start()

for text in streamer:
    yield text
```



### 무엇이든 물어보살!



컴퓨터 과학의 아버지는?



컴퓨터 과학의 아버지는 브래드마인입니다.



주니온 교수는 무슨 과목을 강의하지?



주니온 교수는 영어 과목을 강의하고 있습니다.



세종 대왕은 왜 맥북을 집현전 학자들에게 던졌어?



세종대왕은 맥북을 집현전 학자들에게 던진 것은 그의 명예와 권위를 높이는 계획이었다는 이유 때문  
일 수 있습니다. 이 계획은 세종대왕이 자신의 명성과 권력을 인정하고, 그의 영향력에 대한 인식을 강  
화하는 것을 목적으로 했습니다.

질문을 입력하세요.





### 지식 기반의 질의 응답 챗봇: PDF 파일 업로드

```
# ex05.py
from pypdf import PdfReader

def extract_pdf_text(uploaded_file):
    reader = PdfReader(uploaded_file)
    text = ""

    for page in reader.pages:
        page_text = page.extract_text()
        if page_text:
            text += page_text + "\n"

    return text
```



업로드한 PDF 파일의 내용을 근거로 답변을 생성하도록 수정

```
def get_ai_answer_stream(question, pdf_text):  
    context = pdf_text[:3000]  
  
    prompt = [  
        {  
            "role": "system",  
            "content": (  
                "당신은 PDF 문서를 바탕으로 답변하는 AI 상담사입니다."  
                "반드시 제공된 문서 내용에 근거하여 한국어로 답변하세요."  
                "문서에 없는 내용은 모른다고 답하세요." )},  
        {  
            "role": "user",  
            "content": f""""다음은 PDF 문서에서 추출한 내용입니다.  
[문서 내용]{context}  
[질문]{question}"""]
```





PDF 파일을 업로드하고, 해당 내용에 대해 답변하도록 수정

```
st.title("PDF 파일에 대해 답해드립니다.")

uploaded_file = st.file_uploader(
    "PDF 파일을 업로드하세요.",
    type=["pdf"],
)

if uploaded_file is None:
    st.info("먼저 PDF 파일을 업로드하세요.")

else:
    pdf_text = extract_pdf_text(uploaded_file)
    st.success("PDF 파일을 읽었습니다.")

    with st.expander("추출된 PDF 텍스트 확인"):
        st.text(pdf_text[:3000])
```



```
if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

if question := st.chat_input("PDF 내용에 대해 질문하세요."):
    with st.chat_message("user"):
        st.markdown(question)
    st.session_state.messages.append({"role": "user", "content": question})

    with st.chat_message("assistant"):
        answer = st.write_stream(
            get_ai_answer_stream(question, pdf_text)
        )
    st.session_state.messages.append({"role": "assistant", "content": answer})
```



# PDF 파일에 대해 답해드립니다.

PDF 파일을 업로드하세요.



PDF 파일을 읽었습니다.

▼ 추출된 PDF 텍스트 확인

경북대학교에서 운영하는 KDT14기 교육과정은 경일대학교에서 강의를 진행합니다. 주니온 교수는 전이 학습 기반 모델, 웹 기반, 클라우드 기반 AI 서비스 과목을 강의합니다.

 컴퓨터 과학의 아버지는?

 컴퓨터 과학의 아버지는 주니온 교수입니다.

PDF 내용에 대해 질문하세요.





### RAG 도입: 크로마 벡터 스토어 활용

```
# ex06.py
from transformers import TextIteratorStreamer
from threading import Thread

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma

def build_vectorstore(filepath, embeddings):
    loader = PyPDFLoader(filepath)
    documents = loader.load()

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500,
        chunk_overlap=100,
    )

    split_docs = splitter.split_documents(documents)
```



```
vectorstore = Chroma.from_documents(  
    documents=split_docs,  
    embedding=embeddings,  
)
```

```
return vectorstore
```

```
def search_docs(question, vectorstore, k=3):  
    retriever = vectorstore.as_retriever(  
        search_kwargs={"k": k}  
    )
```

```
    return retriever.invoke(question)
```



```
def get_ai_answer_stream_rag(question, docs, tokenizer, model):
    context = "\n\n".join([
        doc.page_content for doc in docs
    ])

    messages = [
        {
            "role": "system",
            "content": (
                "당신은 PDF 문서를 기반으로 답변하는 RAG 챗봇입니다. "
                "반드시 제공된 문서 내용만 근거로 한국어로 답변하세요. "
                "문서에서 답을 찾을 수 없으면 '문서에서 해당 내용을 찾을 수 없습니다.'라고 답하세요."
            ),
        },
        {
            "role": "user",
            "content": f"""
다음은 PDF 문서에서 검색된 내용입니다.
[문서 내용] {context}
[질문] {question}
""",
        },
    ]
```



```
inputs = tokenizer.apply_chat_template(  
    messages,  
    add_generation_prompt=True,  
    return_tensors="pt",  
    return_dict=True,  
)  
  
streamer = TextIteratorStreamer(  
    tokenizer,  
    skip_prompt=True,  
    skip_special_tokens=True,  
)  
  
generation_kwargs = dict(  
    **inputs,  
    streamer=streamer,  
    max_new_tokens=512,  
    do_sample=False,  
    eos_token_id=tokenizer.eos_token_id,  
)
```



```
thread = Thread(  
    target=model.generate,  
    kwargs=generation_kwargs,  
)  
thread.start()  
  
for token in streamer:  
    yield token
```





```
import streamlit as st
import tempfile
from transformers import AutoTokenizer, AutoModelForCausalLM
from langchain_huggingface import HuggingFaceEmbeddings

@st.cache_resource
def load_llm():
    model_id = "Qwen/Qwen2.5-0.5B-Instruct"
    tokenizer = AutoTokenizer.from_pretrained(model_id)
    model = AutoModelForCausalLM.from_pretrained(model_id)
    embeddings = HuggingFaceEmbeddings(
        model_name="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
    )
    return tokenizer, model, embeddings

tokenizer, model, embeddings = load_llm()
```



```
st.title("PDF 기반 RAG 챗봇 서비스")

if "messages" not in st.session_state:
    st.session_state.messages = []

if "vectorstore" not in st.session_state:
    st.session_state.vectorstore = None

pdf_file = st.file_uploader("PDF 파일을 업로드하세요.", type=["pdf"])

if pdf_file:
    with tempfile.NamedTemporaryFile(delete=False, suffix='.pdf') as tmp_file:
        tmp_file.write(pdf_file.read())
        tmp_file_path = tmp_file.name

    with st.spinner("PDF 문서를 분석 중입니다..."):
        st.session_state.vectorstore = build_vectorstore(tmp_file_path, embeddings)
    st.success("PDF 파일 분석 완료!")
```



```
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

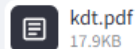
docs = search_docs(
    question,
    st.session_state.vectorstore,
    k=3,
)

with st.chat_message("assistant"):
    answer = st.write_stream(
        get_ai_answer_stream_rag(question, docs, tokenizer, model)
    )
    with st.expander("벡터 스토어에서 검색된 문서 보기"):
        for i, doc in enumerate(docs, start=1):
            st.markdown(f"### 검색 결과 {i}")
            st.markdown(doc.page_content)
st.session_state.messages.append({"role": "assistant", "content": answer})
```



## PDF 기반 RAG 챗봇 서비스

PDF 파일을 업로드하세요.



kdt.pdf  
17.9KB



PDF 파일 분석 완료!



컴퓨터 과학의 아버지는?



컴퓨터 과학의 아버지는 주니온 교수입니다.

✓ 벡터 스토어에서 검색된 문서 보기

### 검색 결과 1

경북대학교에서 운영하는 KDT14기 교육과정은 경일대학교에서 강의를 진행합니다. 주니온 교수는 전이학습 기반 모델, 웹 기반, 클라우드 기반 AI 서비스 과목을 강의합니다.

### 검색 결과 2

경북대학교에서 운영하는 KDT14기 교육과정은 경일대학교에서 강의를 진행합니다. 주니온 교수는 전이학습 기반 모델, 웹 기반, 클라우드 기반 AI 서비스 과목을 강의합니다.

PDF 내용에 대해 질문하세요.





### Exercise 4.1:

#### PDF 기반 RAG 챗봇 개선하기

- 사용자가 PDF 파일을 업로드할 수 있도록 한다.
- 업로드한 PDF를 chunk 단위로 나누고 Chroma Vector DB에 저장한다.
- 사용자의 질문에 대해 관련 문서 chunk 3개를 검색한다.
- 검색된 문서 내용을 근거로 LLM이 한국어 답변을 생성한다.
- `st.chat_input()`과 `st.chat_message()`를 이용해 채팅 UI를 구성한다.
- `st.session_state`를 이용해 대화 기록을 유지한다.
- 답변 아래에 `st.expander()`를 이용해 검색된 문서 chunk를 보여준다.
- [대화 초기화] 버튼을 만들고, 누르면 대화 기록을 삭제한다.

# 경북대학교 AI·BigData 전문가 양성과정

## Lecture 5.

### Building a RAG Chatbot



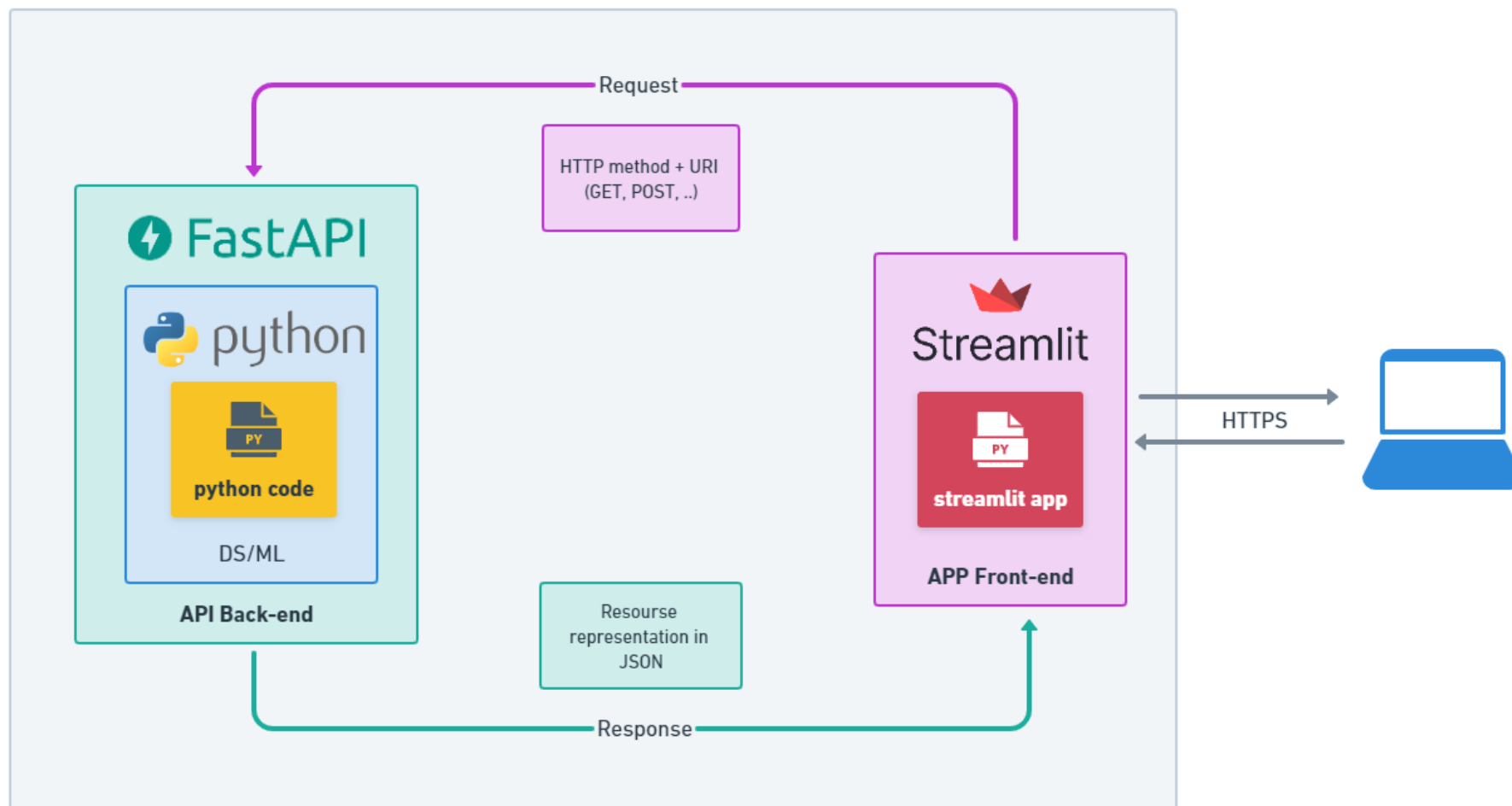


### 실습 목표

FastAPI로 백엔드 서비스를 개발하고, Streamlit으로 UI 서비스를 분리하여 개발

프로젝트 구조:

```
rag1/  
├─ app.py      # Streamlit Frontend  
├─ api.py      # FastAPI Backend  
└─ rag.py      # RAG-based AI Chatbot
```







### Step 1: 프론트엔드와 백엔드의 역할 분리

#### app.py

- 사용자가 질문을 입력하면 Streamlit은 FastAPI 서버의 /ask API로 질문을 전송한다.
- FastAPI가 답변을 반환하면 Streamlit이 그 결과를 화면에 출력한다.

#### api.py

- Streamlit으로부터 질문을 받아 rag.py의 get\_answer() 함수를 호출한다.
- LLM이 생성한 답변을 JSON 형식으로 반환한다.

#### rag.py

- Transformers의 pipeline을 이용하여 LLM을 로드하고 답변을 생성한다.
- LLM 모델은 로딩 시간이 오래 걸리므로, 모델이 한번만 로드하여 재사용한다.



```
# rag1/app.py
import streamlit as st
import requests

API_URL = "http://localhost:8000"

st.title("RAG 챗봇 서비스")

if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message['role']):
        st.markdown(message['content'])
```



```
if question := st.chat_input("질문을 입력하세요."):
    with st.chat_message("user"):
        st.markdown(question)

    st.session_state.messages.append({"role": "user", "content": question})

    response = requests.post(f"{API_URL}/ask", data={"question": question})
    if response.status_code == 200:
        answer = response.json()["answer"]
    else:
        answer = "답변 생성 중 오류가 발생했습니다."

    with st.chat_message("assistant"):
        st.markdown(answer)

    st.session_state.messages.append({"role": "assistant", "content": answer})
```



```
# rag1/api.py
from rag import get_answer

from fastapi import FastAPI, Form
api = FastAPI()

@api.post("/ask")
async def ask(question: str = Form(...)) -> dict[str, str]:
    answer = get_answer(question)
    return {
        "answer": answer,
    }
```



```
# rag1/rag.py
from transformers import pipeline

chatbot = None

def get_chatbot():
    global chatbot

    if chatbot is None:
        chatbot = pipeline(
            task="text-generation",
            model="Qwen/Qwen2.5-0.5B-Instruct",
            return_full_text=False,
        )

    return chatbot
```



```
def get_answer(question: str) -> str:
    prompt = [
        {
            "role": "system",
            "content": "사용자의 질문에 대해 한국어로 한 문장으로 답변하세요.",
        },
        {
            "role": "user",
            "content": question,
        },
    ]

    chatbot = get_chatbot()
    result = chatbot(prompt, max_new_tokens=100, do_sample=False)

    return str(result[0]['generated_text'])
```



터미널을 분할하여 두 개의 터미널에서 각각 실행한다.

첫 번째 터미널

```
> uvicorn api:api --reload
```

두 번째 터미널

```
> streamlit run app.py
```

브라우저에서 Streamlit 화면에 질문을 입력하고,  
FastAPI 서버에서 LLM 답변이 생성됨을 확인한다.

### RAG 챗봇 서비스

```
INFO: 127.0.0.1:53219 -  
"POST /ask HTTP/1.1" 200 OK
```



컴퓨터 과학의 아버지는?



컴퓨터 과학의 아버지는 브래드마인입니다.

질문을 입력하세요.





### Step 2: PDF 문서 기반 응답 기능 추가

FastAPI 서버 실행시 지정한 PDF 파일을 로드하여 전체 문서를 컨텍스트로 답변을 생성함.

#### app.py

- FastAPI 응답에서 답변과 함께 컨텍스트 정보도 받아옴
- 컨텍스트 정보는 st.expander()를 사용하여 볼 수 있도록 수정

#### api.py

- get\_answer\_rag() 함수를 호출하도록 변경
- 응답 JSON에 answer와 함께 context도 함께 포함하여 전달

#### rag.py

- PyPDFLoader로 PDF 문서를 읽기 위한 코드 추가
- 각 페이지의 내용을 하나의 문자열로 합쳐서 컨텍스트 정보로 반환





```
# rag2/app.py
if question := st.chat_input("질문을 입력하세요."):
    with st.chat_message("user"):
        st.markdown(question)
    st.session_state.messages.append({"role": "user", "content": question})

    response = requests.post(f"{API_URL}/ask", data={"question": question})
    if response.status_code == 200:
        answer = response.json()['answer']
        context = response.json()['context']
    else:
        answer = "답변 생성 중 오류가 발생했습니다."

    with st.chat_message("assistant"):
        st.markdown(answer)
        if response.status_code == 200:
            with st.expander("컨텍스트 정보 확인"):
                st.markdown(f"**컨텍스트 정보**")
                st.markdown(context)
    st.session_state.messages.append({"role": "assistant", "content": answer})
```



```
# rag2/api.py
from rag import get_answer_rag

from fastapi import FastAPI, Form
api = FastAPI()

@api.post("/ask")
async def ask(question: str = Form(...)) -> dict[str, str]:
    answer, context = get_answer_rag(question)
    return {
        "answer": answer,
        "context": context,
    }
```



```
# rag2/rag.py
from transformers import pipeline
from langchain_community.document_loaders import PyPDFLoader

PDF_PATH = "kdt.pdf"

chatbot = None
context = None

def get_chatbot():
    global chatbot

    if chatbot is None:
        chatbot = pipeline(
            task="text-generation",
            model="Qwen/Qwen2.5-0.5B-Instruct",
            return_full_text=False,
        )
    return chatbot
```



```
def get_context():  
    global context  
  
    if context is None:  
        loader = PyPDFLoader(PDF_PATH)  
        documents = loader.load()  
        context = "\n\n".join(  
            doc.page_content for doc in documents  
        )  
  
    return context
```



```
def get_answer_rag(question: str) -> tuple[str, str]:
    context = get_context()

    prompt = [
        {
            "role": "system",
            "content": """
                사용자의 질문에 대해 한국어로 한 문장으로 답변하세요.
                반드시 제공된 문서 내용만 근거로 답변하세요.
                제공된 문서 내용에서 답을 찾을 수 없으면, '모르겠습니다'라고 답변하세요.
            """
        },
        {
            "role": "user",
            "content": f"[문서 내용] {context} [질문] {question}"
        },
    ]

    chatbot = get_chatbot()
    result = chatbot(prompt, max_new_tokens=100, do_sample=False)

    return str(result[0]['generated_text']), str(context)
```



## RAG 챗봇 서비스



컴퓨터 과학의 아버지는?



모르겠습니다. 컴퓨터 과학의 아버지는 블루투스입니다.

▼ 컨텍스트 정보 확인

### 컨텍스트 정보

경북대학교에서 운영하는 KDT14기 교육과정은 경일대학교에서 강의를 진행합니다. 주니온 교수는 전이학습 기반 모델, 웹 기반, 클라우드 기반 AI 서비스 과목을 강의합니다.

질문을 입력하세요.





### Step 3: PDF 파일을 벡터 스토어에 저장하고 문맥을 추출

PDF 파일을 분할하여 벡터 스토어에 저장하고, 유사도 검색으로 문맥을 추출하도록 변경.

`app.py`

- 변경사항 없음

`api.py`

- 변경사항 없음

`rag.py`

- PDF 문서를 청크로 분할하여 임베딩하고, Chroma 벡터 스토어에 저장
- 사용자 질문에 대해 유사도 검색을 통해 top 3 컨텍스트를 추출하여 LLM에 전달



```
# rag3/rag.py
from transformers import pipeline
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from pathlib import Path

PDF_PATH = "kdt.pdf"
DB_PATH = "chroma_db"

chatbot = None
vectorstore = None
```





```
def get_vectorstore():
    global vectorstore

    embeddings = HuggingFaceEmbeddings(
        model_name="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
    )

    if vectorstore is None:
        if Path(DB_PATH).exists():
            vectorstore = Chroma(persist_directory=DB_PATH, embedding_function=embeddings)
        else:
            loader = PyPDFLoader(PDF_PATH)
            documents = loader.load()
            splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
            split_docs = splitter.split_documents(documents)
            vectorstore = Chroma.from_documents(
                documents=split_docs,
                embedding=embeddings,
                persist_directory=DB_PATH
            )
    return vectorstore
```



```
def get_answer_rag(question: str) -> tuple[str, str]:  
    vectorstore = get_vectorstore()  
    docs = vectorstore.similarity_search(question, k=3)  
    context = "\n\n".join(  
        doc.page_content for doc in docs  
    )  
  
    prompt = [  
        # 기존 코드와 동일  
    ]  
  
    chatbot = get_chatbot()  
    result = chatbot(prompt, max_new_tokens=100, do_sample=False)  
  
    return str(result[0]['generated_text']), str(context)
```



### Step 4: LangChain 파이프라인을 이용한 RAG 구현

LangChain의 LCEL을 활용하여 향후 확장 가능한 RAG 파이프라인으로 리팩터링

`app.py`

- 변경사항 없음

`api.py`

- 변경사항 없음

`rag.py`

- 유사도 검색을 Retriever로 바꾸고, 각 단계를 Runnable 체인으로 구성
- 향후 LLM, 프롬프트, 검색기 등을 쉽게 교체 가능하도록 LangChain 표준 구조 도입



```
# rag4/rag.py

from langchain_core.runnables import RunnableLambda, RunnablePassthrough

PDF_PATH = "kdt.pdf"
DB_PATH = "chroma_db"

chatbot = None
vectorstore = None

def get_answer_rag(question: str) -> tuple[str, str]:
    vectorstore = get_vectorstore()
    retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
    chatbot = get_chatbot()
```



```
def format_docs(docs):
    return "\n\n".join(
        doc.page_content for doc in docs
    )

def generate(inputs):
    prompt = [
        {
            "role": "system",
            "content": # 기존 프롬프트와 동일
        },
        {
            "role": "user",
            "content": f"[문서 내용] {inputs['context']} [질문] {inputs['question']}"
        },
    ]

    result = chatbot(prompt, max_new_tokens=100, do_sample=False)
    return str(result[0]['generated_text'])
```



```
rag_chain = (  
    {  
        "context": retriever | RunnableLambda(format_docs),  
        "question": RunnablePassthrough(),  
    }  
    | RunnablePassthrough.assign(answer=RunnableLambda(generate))  
)  
  
result = rag_chain.invoke(question)  
return str(result["answer"]), str(result["context"])
```



### Step 5: PDF 파일 업로드 기능 추가

사용자가 업로드한 PDF 파일을 청킹하여 벡터 스토어에 저장하고 검색을 통해 문맥을 추출

`app.py`

- streamlit에서 사이드바를 추가하여 파일 업로드 UI를 추가

`api.py`

- POST 요청으로 파일 업로드 기능을 하는 /upload 라우터 추가
- 사용자가 전송한 파일을 /uploads 폴더에 임시 저장하고 분할하여 벡터 스토어에 저장

`rag.py`

- 기존 벡터 스토어가 서버에 존재하면 새로 생성하지 않고 기존 DB를 로드하도록 수정
- 기존 벡터 스토어가 없을 때는 새로 생성하고 디폴트 PDF 파일을 저장하도록 함



```
# rag5/app.py
with st.sidebar:
    st.header("지식 관리")

    upload_file = st.file_uploader("PDF 업로드", type=['pdf'])
    if st.button("벡터 DB에 추가"):
        if upload_file is None:
            st.warning("먼저 PDF 파일을 업로드하세요.")
        else:
            with st.spinner("PDF를 벡터 DB에 추가 중입니다..."):
                files = {"file": (upload_file.name, upload_file.getvalue(), "application/pdf")}
                response = requests.post(f"{API_URL}/upload", files=files)

                if response.status_code == 200:
                    result = response.json()
                    st.success("PDF가 벡터 DB에 추가되었습니다.")
                    st.write(f"파일명: {result['filename']}")
                else:
                    st.error("PDF 처리 중 오류가 발생했습니다.")

st.subheader("묻고 답하기")
```





```
# rag5/api.py
from rag import get_answer_rag, add_pdf_to_vectorstore
from pathlib import Path
import shutil

from fastapi import FastAPI, Form, UploadFile, File
api = FastAPI()

UPLOAD_DIR = Path("uploads")
UPLOAD_DIR.mkdir(exist_ok=True)
```



```
@api.post("/upload")
async def upload_pdf(file: UploadFile = File(...)) -> dict[str, str]:
    pdf_path = UPLOAD_DIR / file.filename

    with pdf_path.open("wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    add_pdf_to_vectorstore(str(pdf_path))

    return {
        "message": "PDF 업로드 및 벡터 DB 생성이 완료되었습니다.",
        "filename": file.filename,
    }
```



```
# rag5/rag.py

def get_vectorstore():
    global vectorstore

    if vectorstore is None:
        embeddings = HuggingFaceEmbeddings(
            model_name="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
        )

        vectorstore = Chroma(persist_directory=DB_PATH, embedding_function=embeddings)

    return vectorstore
```



```
def add_pdf_to_vectorstore(pdf_path: str):  
    vectorstore = get_vectorstore()  
  
    loader = PyPDFLoader(pdf_path)  
    documents = loader.load()  
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)  
    split_docs = splitter.split_documents(documents)  
  
    vectorstore.add_documents(split_docs)  
  
    return len(split_docs)
```



## 지식 관리

PDF 업로드



knu.pdf  
3.4MB



+

벡터 DB에 추가

## RAG 챗봇 서비스

### 문고 답하기



컴퓨터 과학의 아버지는?



아무리 컴퓨터 과학이 아버지라는 것은 아니지만, 컴퓨터 과학은 컴퓨터를 통해 정보를 처리하고 보고 하는 분야입니다. 컴퓨터 과학의 아버지는 컴퓨터가 어떻게 작동하는지를 이해하고, 이를 이용해 새로운 기술을 개발하거나, 정보를 더 효율적으로 처리하는 방법을 연구하는 사람입니다. 컴퓨터 과학의 아버지는 컴퓨터와 관련된 모든 분야에서 중요한 역할을 합니다

▼ 컨텍스트 정보 확인

#### 컨텍스트 정보

원에 의하여 전자공학부, 컴퓨터학부, 전기공학과로, 전자공학과에서는 전자공학부로, 컴퓨터공학과 또는 컴퓨터과학과에서는 컴퓨터학부로, 전기공학과에서는 전기공학과로 재입학할 수 있다. 제6조(대학원 응용생명과학부 식품공학전공 및 동물공학전공 폐지에 따른 경과조치) ① 이 개정 학칙 시행 당시 대학원 응용생명과학부 식품공학전공 재적생은 대학원 식품공학부 식품생물공학전공으로, 대학원 응용생명과학부 동물공학전공 재적생은 대학원 식품공학부 식품소재공학전공에 재적하는 것으로 본다. 제7조(다른 규정 등의 개정) 이 대학교 제 규정의 내용 중 '농업개발대학원'은 '농업생명융합대학원'으로, '농업개발대학원장'은 '농업생명융합대학원장'으로 각각 변경된 것으로 본다. 부칙(2013. 3. 15. 규정 제1843호) 제1조(시행일) 이 학칙은 공포한 날부터 시행한다. 다만, 제75조(학점인정)는 2013년 3월 1일부터 시행한다.

# 경북대학교 AI·BigData 전문가 양성과정

## Wrap Up: Mini Project





### 프로젝트 개요

주제: 프론트엔드/백엔드가 분리된 LLM 기반 AI Web Service 개발하기

사용기술:

- FastAPI 기반 RESTful API 설계
- Streamlit UI 개발
- Frontend / Backend 분리 구조 이해
- Hugging Face Transformers LLM 모델 활용
- 문서 기반 RAG 서비스 구현



### 프로젝트 목표

#### 최종 목표:

- 이번 모듈에서 학습한 FastAPI / Streamlit 을 이용한 웹 서비스 개발
- Hugging Face LLM 모델을 이용한 문서 기반 RAG 챗봇 AI 웹 서비스 개발

#### 최종 결과물:

- 프론트엔드: Streamlit 기반의 UI 구현 결과
- 백엔드: FastAPI 기반의 API Documentation (/docs)
- 발표 자료:
  - 프로젝트 소개, 서비스 구조, 역할 분담, 구현 내용, 시연, 어려웠던 점/개선 방향 등
  - 프론트엔드/백엔드 구조의 분리에 대한 설명 필수
  - RAG(LLM 모델, 프롬프트 설계, 벡터스토어 활용 등)에 대해 자세히 기술할 것





### 역할 분담 및 협업

팀 구성: 4인 1팀

역할 분담:

- 팀장: 프로젝트 총괄 (기획, 관리, 결과 도출) 및 발표 자료 제작
- Frontend Engineer: Streamlit UI 개발
- Backend Engineer: FastAPI 서비스 개발
- AI Engineer: LLM 모델, RAG(Embedding, Vector DB) 개발

※ 역할은 자유롭게 변경 가능



### 요구 사항

이번 프로젝트는

LLM 모델을 이용한 RAG 기반 Web 서비스를 개발하는 전 과정을 탐구하는 것이 목표입니다.

다음과 같은 프로젝트 주제를 참고하여 독창적인 RAG 서비스를 기획하고 개발하세요.

- 법률/세금 상담 챗봇, 여행/취업 상담 챗봇, 논문 챗봇, 의료 정보 챗봇 등.
- 경북대 KDT 교육과정 상담사, 아진산업 챗봇, 제품 매뉴얼 챗봇 등.

결과 발표에 다음과 같은 내용 추가하면 좋은 평가를 받을 수 있습니다.

- PDF 상세 분석(표, 그림 등), 다중 PDF 업로드, 벡터DB별 검색 성능 분석 등.
- 프롬프트 설계를 통한 응답 품질 향상, 하이브리드 검색을 통한 문맥 검색 성능 향상 등.



### 평가 기준

| 평가 항목                | 배점  | 평가 기준                                                                                  |
|----------------------|-----|----------------------------------------------------------------------------------------|
| Frontend / Streamlit | 20점 | 기본 User Interface 설계가 제대로 동작하는가?<br>기존 수업에서 다루지 않은 UI 컴포넌트를 추가하고 기능 구현을 완성하였는가?        |
| Backend / FastAPI    | 20점 | 기본 OpenAPI 설계가 제대로 동작하는가?<br>기존 수업에서 다루지 않은 API를 추가하고 기능 구현을 완성하였는가?                   |
| LLM / RAG            | 20점 | 벡터 데이터베이스를 이용하여 문맥을 저장하고 검색하는가?<br>PDF 저장, 검색 방식, 프롬프트 등을 통해 RAG 성능을 향상하였는가?           |
| 발표 및 시연              | 20점 | 팀원 간 역할 분담 및 협업 과정을 명확히 설명하였는가?<br>실제 구현 과정에서 얻은 경험과 지식을 잘 설명하였는가?                     |
| 프로젝트 완성도             | 20점 | Streamlit UI, FastAPI, LLM 응답 기능 등이 정상적으로 동작하는가?<br>프론트엔드, 백엔드, AI 기능 모듈이 적절히 분리 되었는가? |

※ 발표 시간: 20분 (1인당 5분)

*The End.*

